MoveIt2 Configuration

> [!IMPORTANT]
> **Important Notes**:
> **The factory system image already includes and pre-generates the matching MoveIt2 package (`dofbot_pro_moveit`), so no re-configuration is needed!**
> It is strongly recommended to first experience the features covered in the following chapters (e.g., MoveIT2 simulation-reality linkage, Random movement, Forward/Inverse kinematics design, Cartesian path , and obstacle-avoidance). Only if you later wish to do your own secondary development (for example, customizing the robot arm URDF model, modifying joint limits, or adding new planning groups) should you refer back to this tutorial section to re-configure.

Preface: The Raspberry Pi 5 runs in Docker, so the MoveIt2 experience is limited; it is recommended that Raspberry Pi 5 users run MoveIt2-related cases on a virtual machine. The Orin board runs directly on the host, so Orin board users can run MoveIt2-related cases directly on the board, using the same commands as running on a virtual machine. This section will use running on a virtual machine as the example for demonstration.

1. Content Description

This section explains how to configure MoveIt2 and generate the parameter files and launch files required for simulation.

2. MoveIt2 Introduction

MoveIt 2 is the next-generation version of **MoveIt**, designed specifically for **ROS 2 (Robot Operating System 2)**. It is an open-source framework for robot motion planning, manipulation, 3D perception, kinematics, collision detection, and task planning. It is widely used in industrial robots, service robots, mobile manipulators, and scientific research and education.

2.1. Differences from MoveIt

Feature	MoveIt	MoveIt2
ROS Version	ROS1	**ROS2**
Real-time	Limited support	**Better real-time performance**
Cross-platform Support	Mainly Linux	**Supports Linux, Windows, macOS** (ROS 2 feature)
Motion Planning Interface	ActionLib (ROS 1)	**Action (ROS 2)**
Default Planner	OMPL	**OMPL + more optimization options**

2.2. MoveIt2 Core Features

- **Motion Planning**: Supports multiple planning algorithms (such as OMPL, CHOMP, STOMP, etc.) to generate collision-free motion trajectories.
- **Inverse Kinematics (IK)**: Computes the robot joint angles to achieve a target pose.
- **Collision Checking**: Performs real-time collision detection based on **FCL (Flexible Collision Library)**.
- **3D Perception**: Supports point cloud (PointCloud) and depth camera (e.g., RealSense, Kinect) data.
- **Manipulation**: Provides high-level interfaces for grasping and placement.
- **Task Planning**: Combines **Behavior Trees** for high-level task orchestration.

2.3. Installing MoveIt2

MoveIt2 is already installed in the virtual machine, so no installation is needed here. If MoveIt2 needs to be installed in a new environment, enter the following command to install it:

```
# Change <distro> below to your own ROS version; here we recommend using humble (22.04)
sudo apt install ros-<distro>-moveit -y
```

2.4. Reference Websites

MoveIt2 source code:

GitHub - moveit/moveit2: MoveIt for ROS 2

Official tutorial documentation:

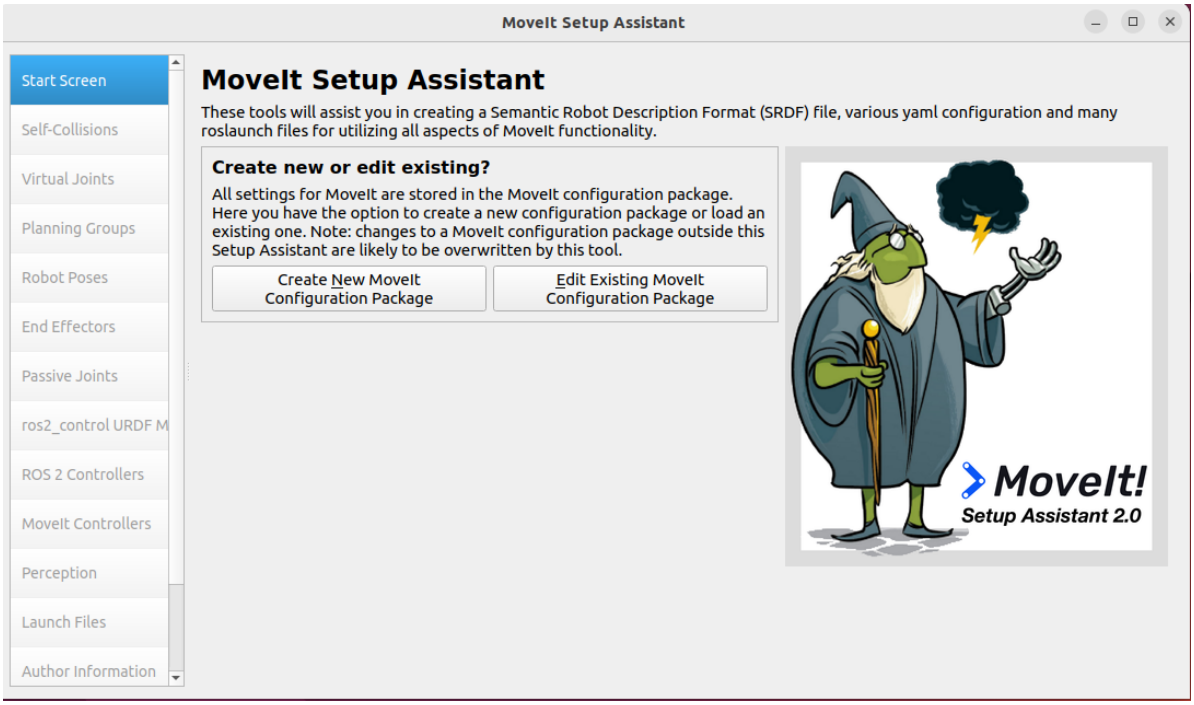
MoveIt 2 Documentation — MoveIt Documentation: Humble documentation

3. Launching the Program

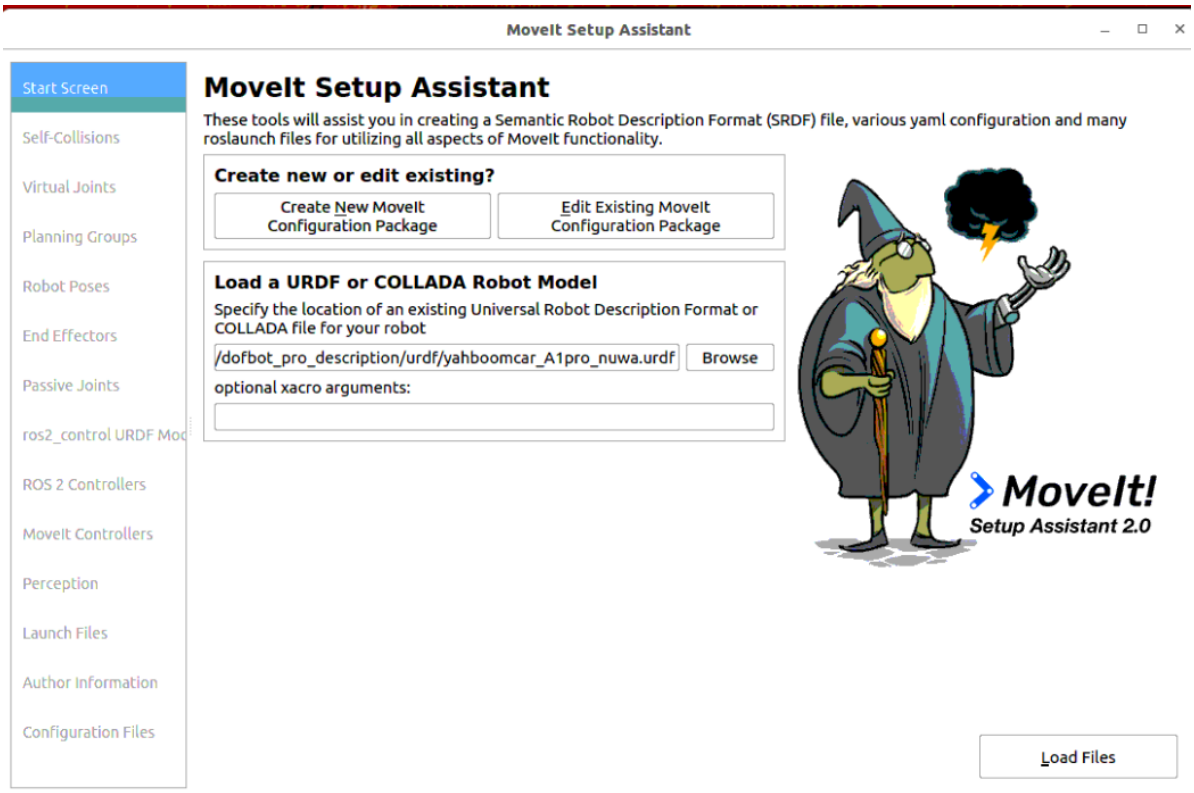
Enter the following command in the terminal to start MoveIt2 configuration:

```
ros2 run moveit_setup_assistant moveit_setup_assistant
```

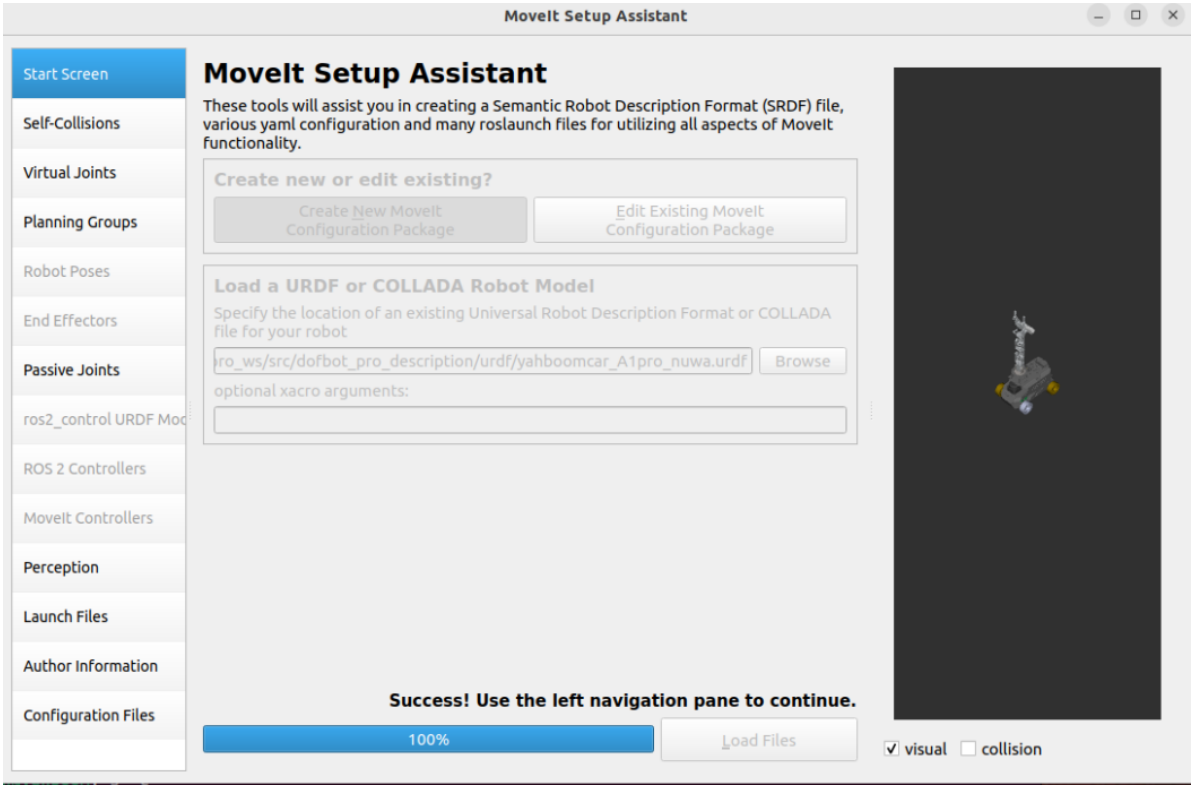
After the program starts, the following configuration screen appears:



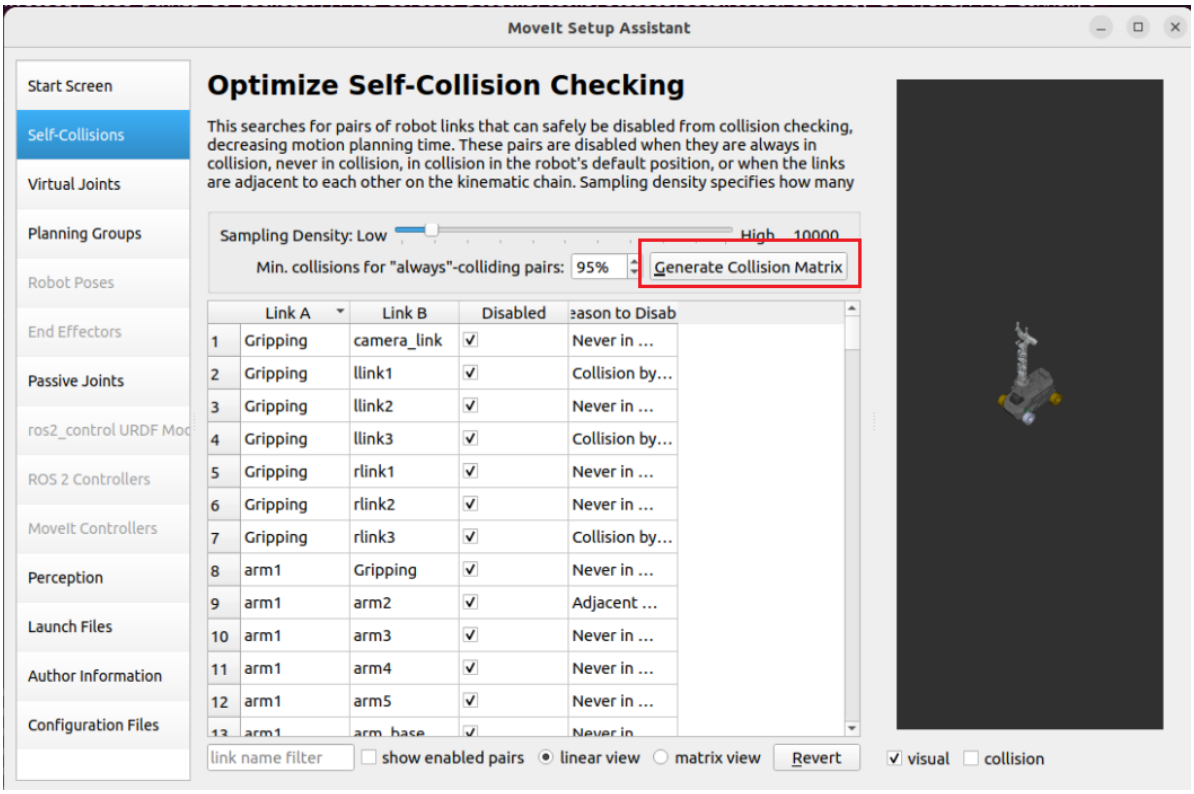
Click 【Create New MoveIt Configuration Package】, and the following screen appears:



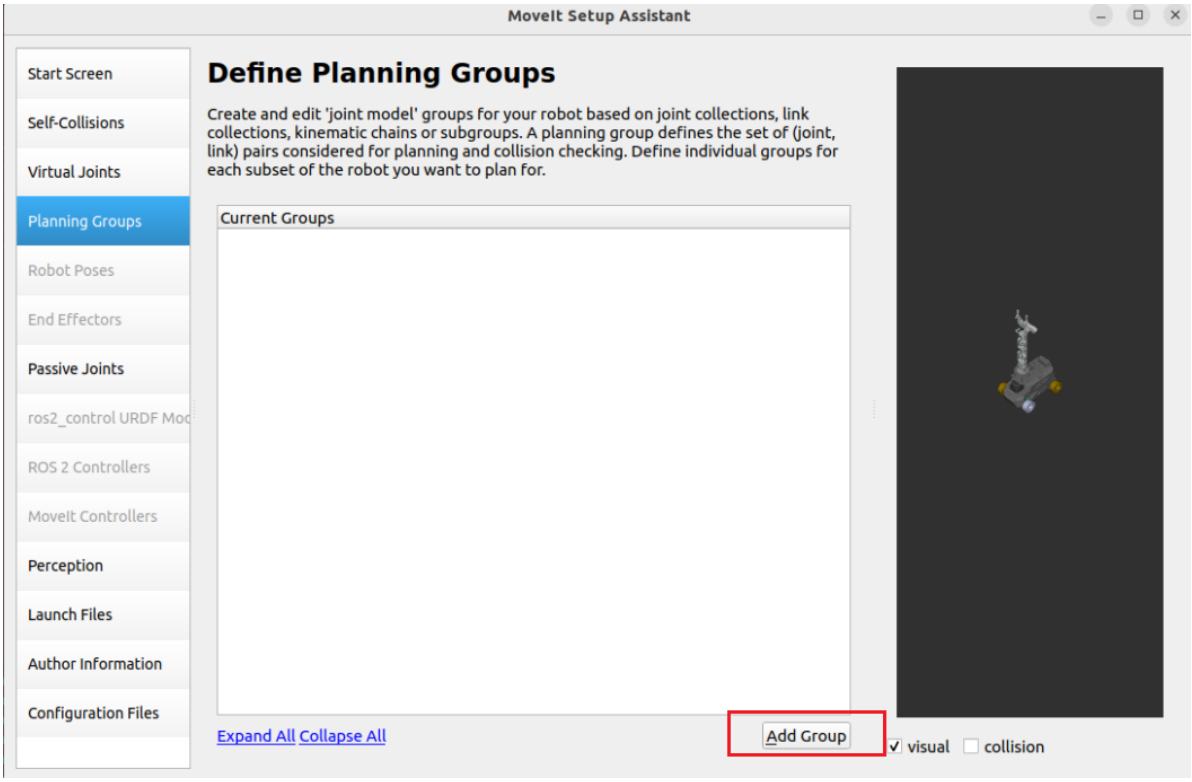
Click 【Browse】, then select the A1Pro URDF file. The file directory is `/home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_description/urdf/yahboomcar_A1pro_nuwa.urdf`. After the selection is complete, click 【Load Files】. The program will then load the selected URDF file. When loading is complete, the following screen appears, with the loaded A1Pro car model displayed on the right.



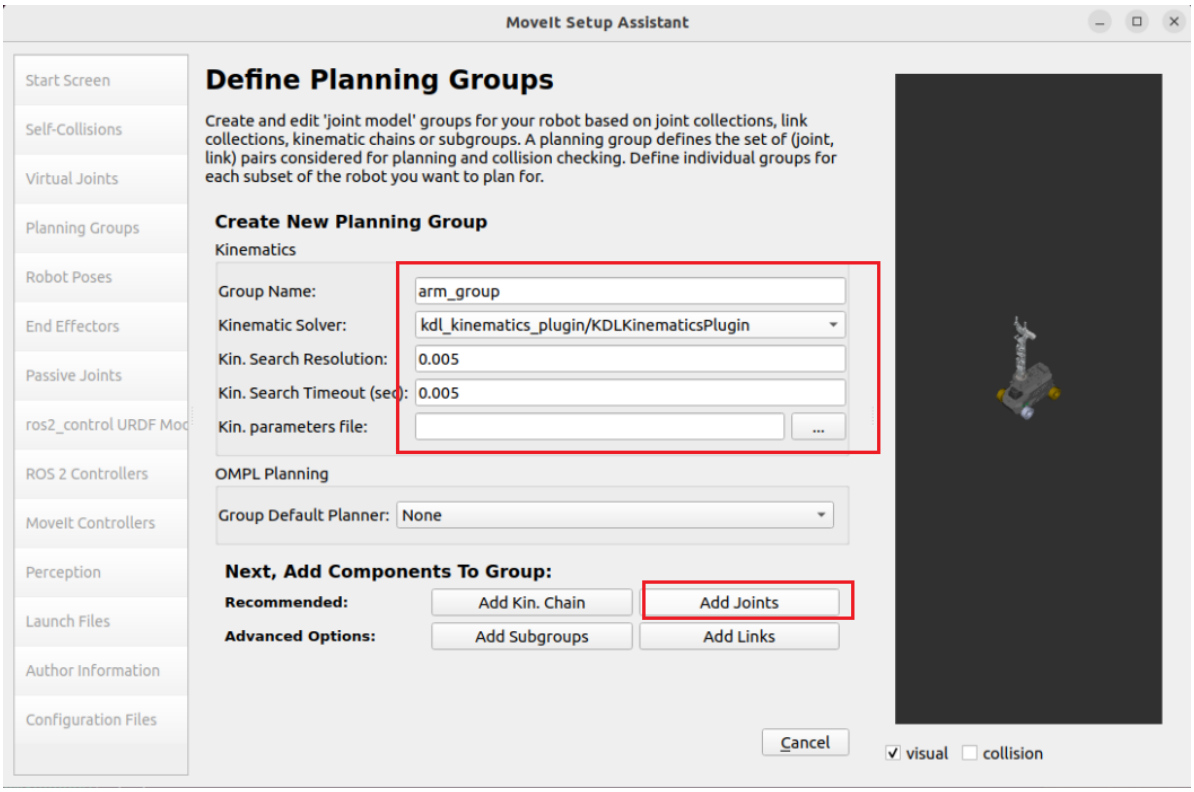
Next, click 【Self-Collisions】 in the configuration bar on the left. This step is to configure self-collision detection. As shown in the figure below, click 【Generate Collisions Matrix】 to complete the configuration:



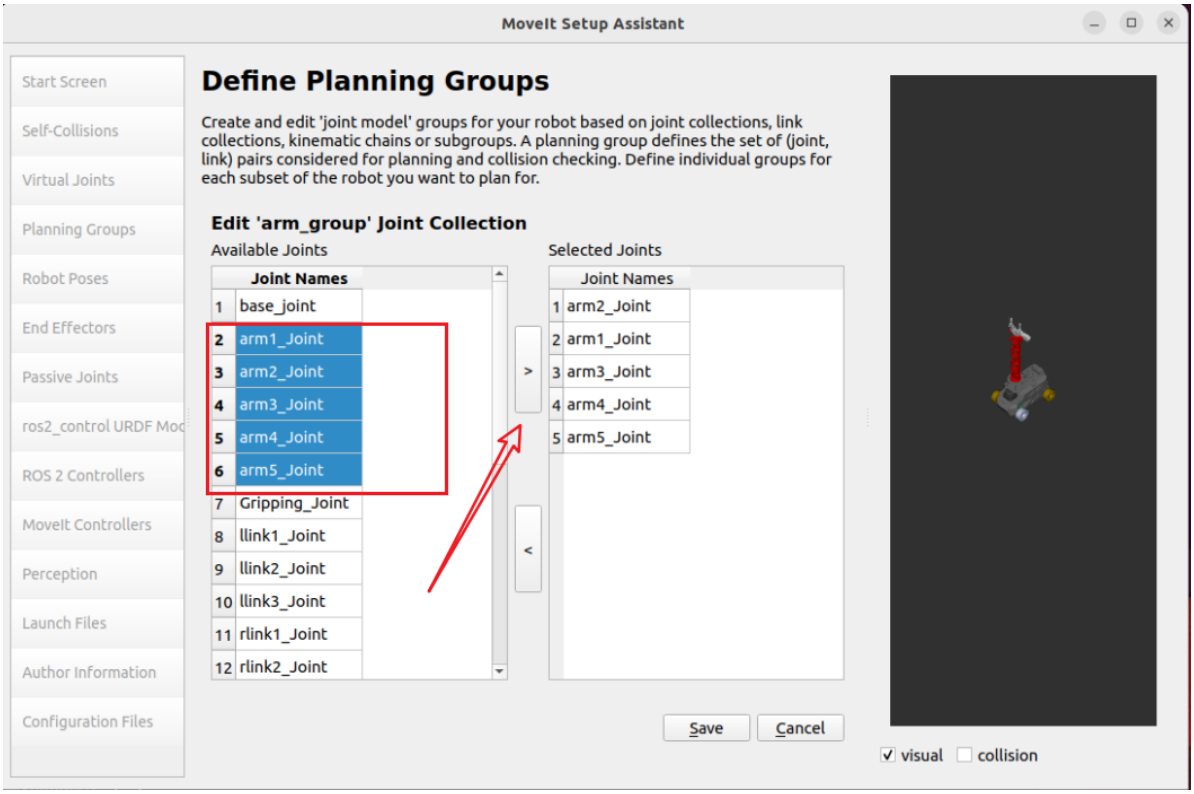
Then, click 【Planning Groups】 in the configuration bar on the left. This step is to create planning groups. Click 【Add Group】 to create the first planning group, as shown in the figure below:



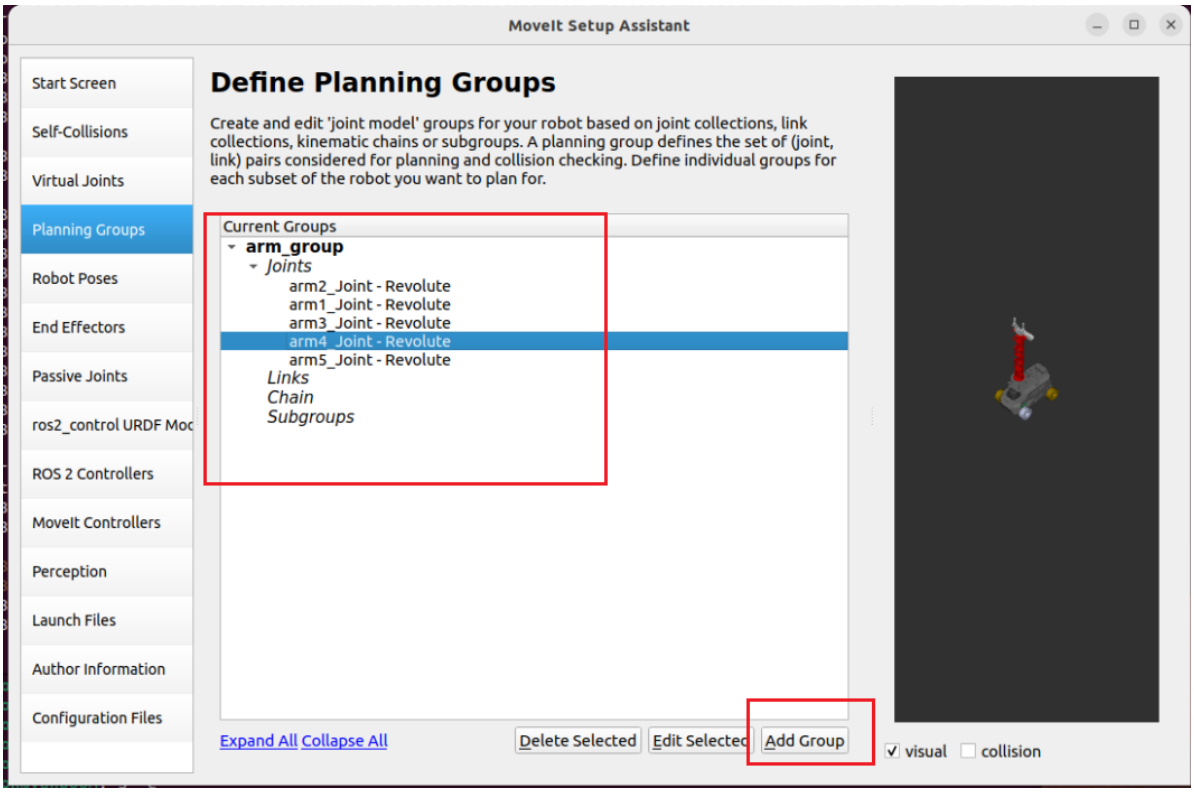
Then create the arm_group planning group to control the robot arm, and set it as shown below:



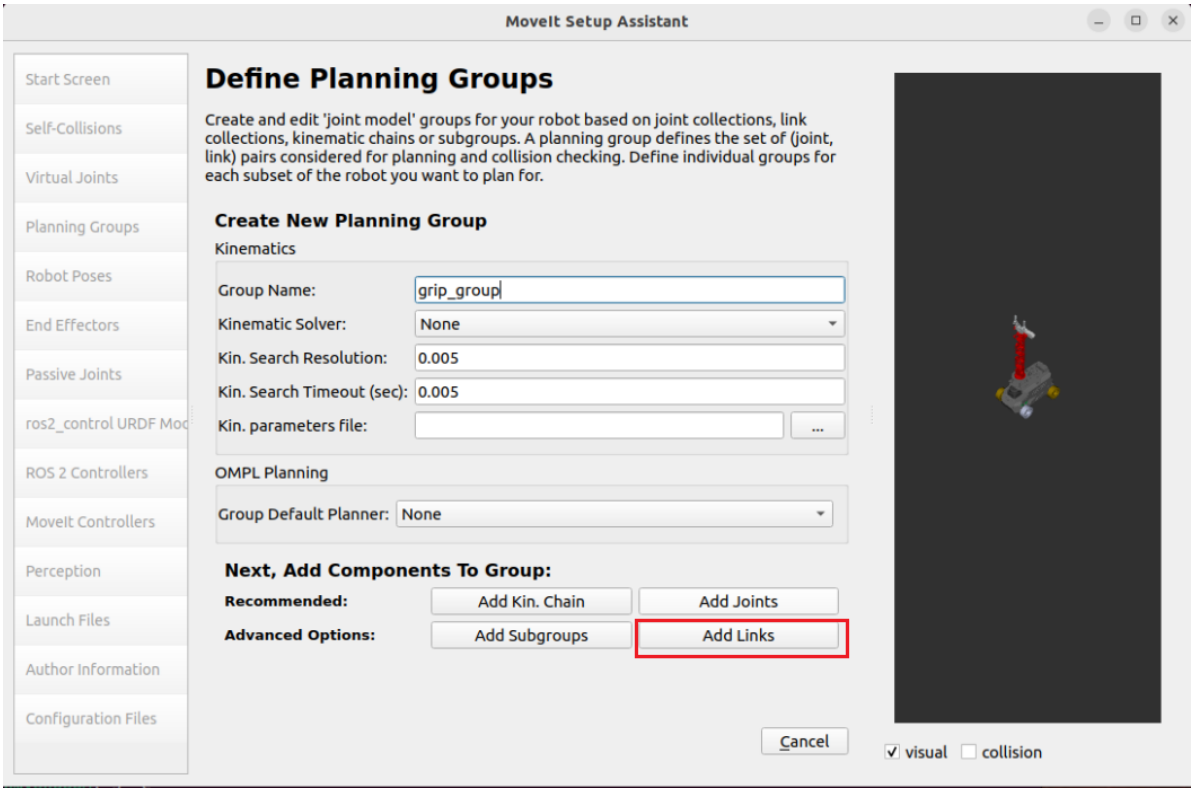
Next, we need to add joints to this planning group. Click 【Add Joints】 below to enter the add interface. In the add interface, select arm_Joint1 through arm_Joint5 on the left, click > to finish adding them, and then click 【Save】 to save and exit.



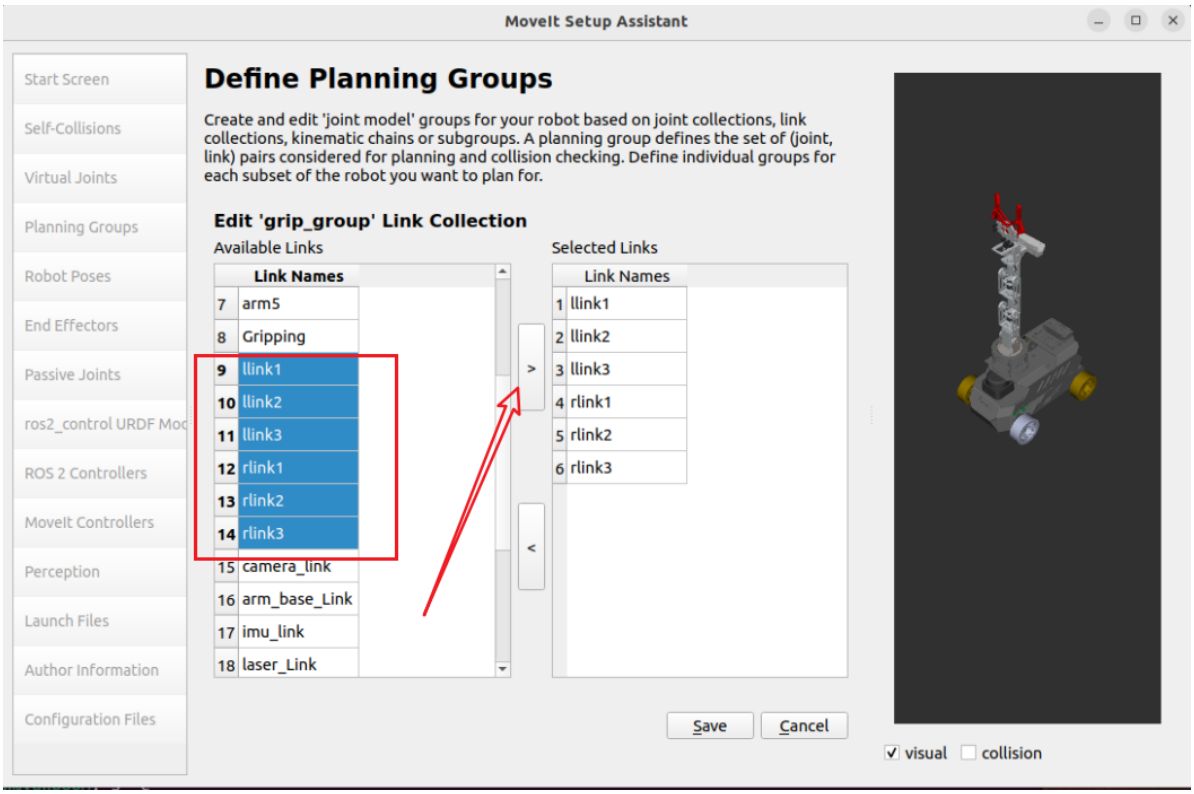
The completed arm_group planning group setup is shown in the figure below. Then click 【Add Group】 to add the gripper planning group:



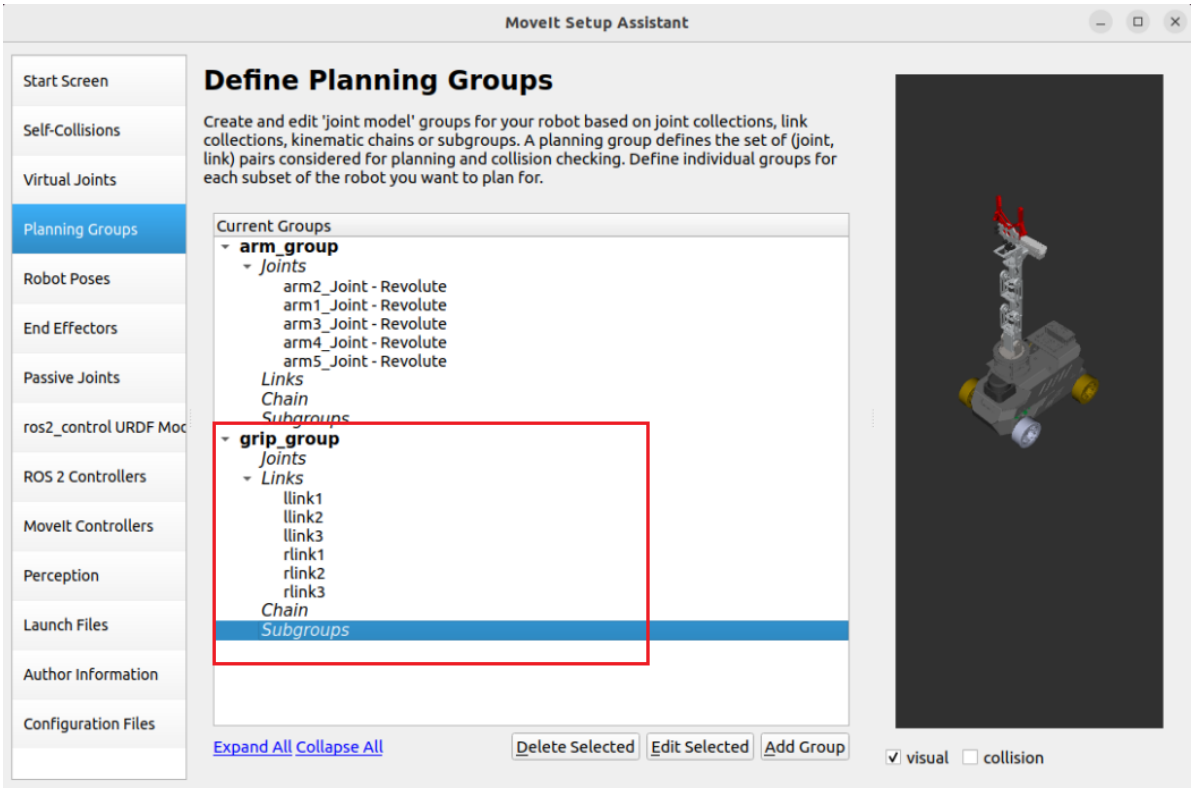
Set the parameters as shown below, then click 【Add Links】 to enter the Link selection interface:



In the Link selection interface, select llink1, llink2, llink3, rlink1, rlink2, and rlink3 on the left, click > to finish adding them, and then click 【Save】 to save and exit.

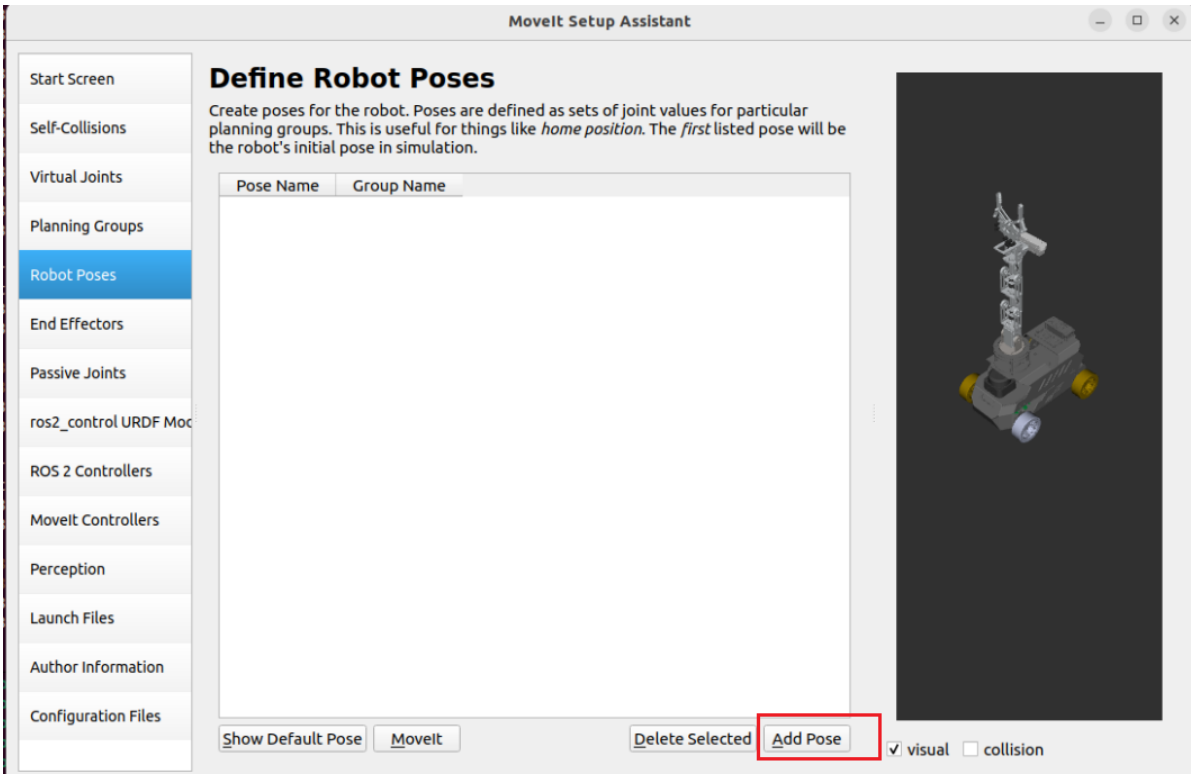


The completed gripper planning group is shown in the figure below:

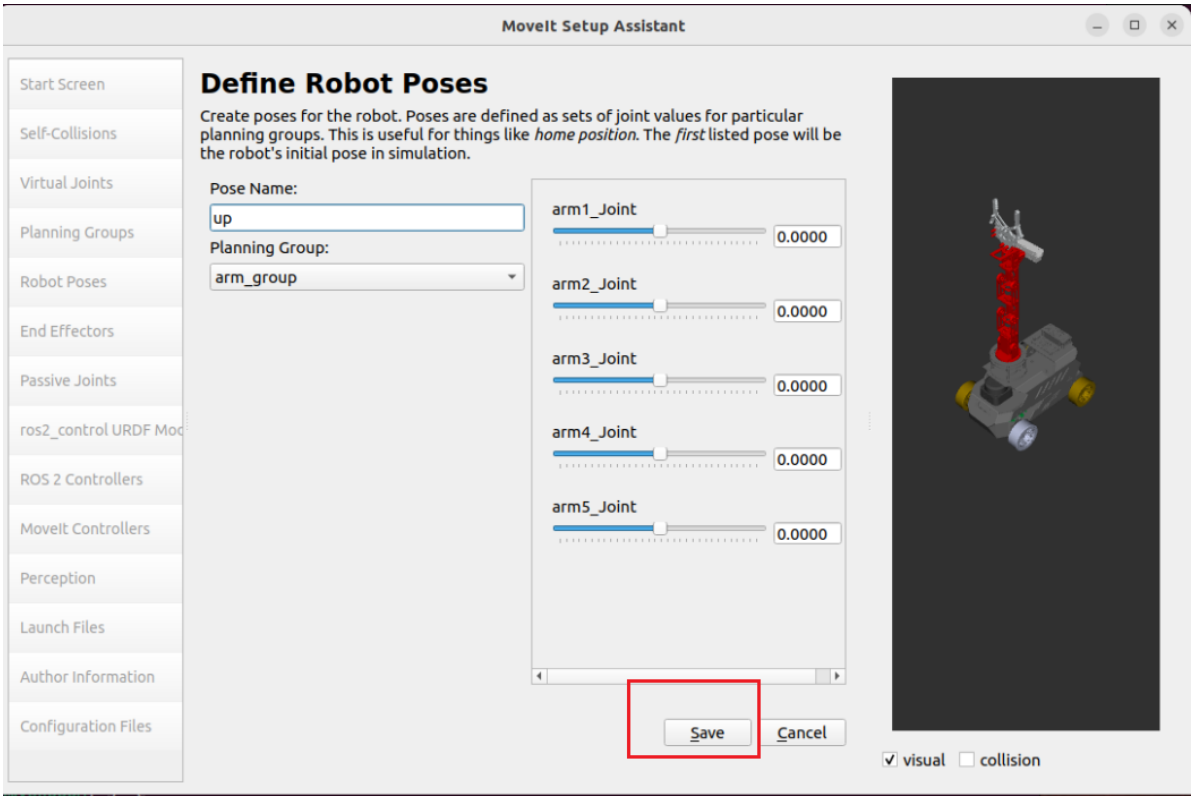


At this point, the planning groups are set up. Next, click 【Robot Groups】 in the settings bar on the left. This step is to set the predefined poses of the robot arm and gripper:

As shown in the figure below, click 【Add Pose】:

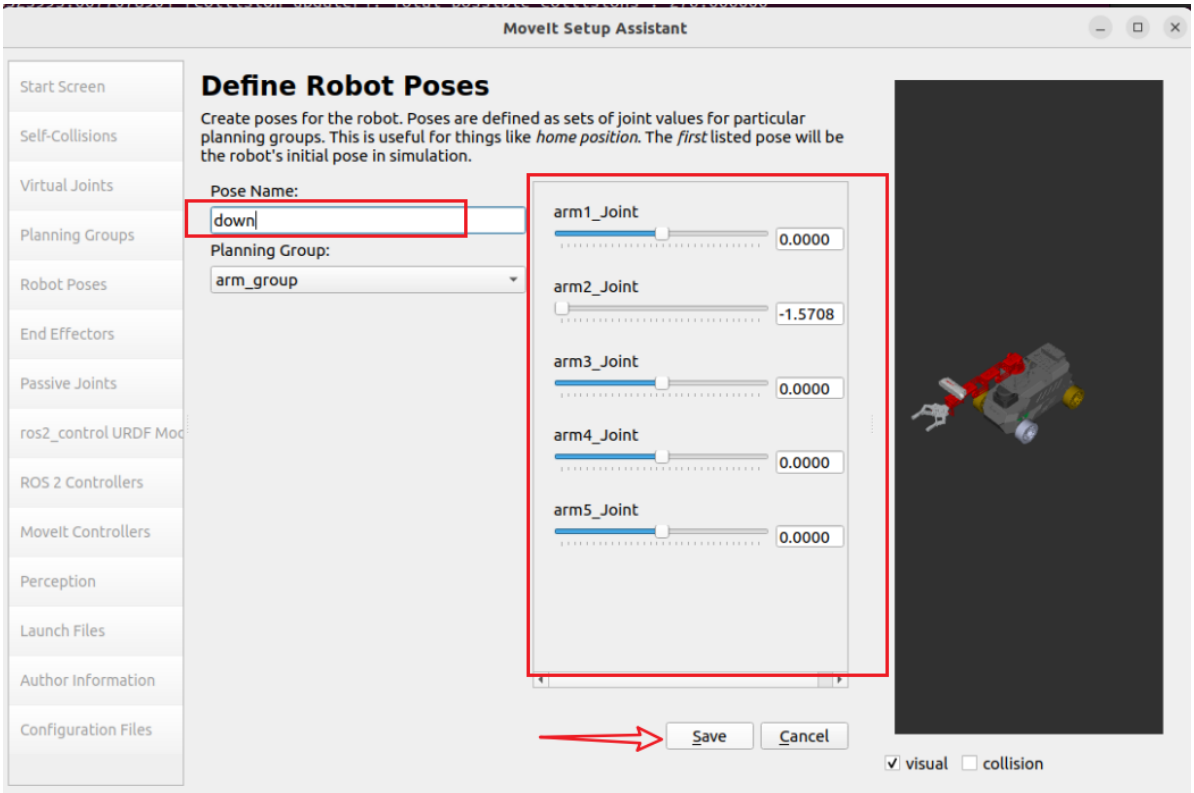


Then configure it as shown below to set the up pose of the arm_group planning group. The slider in the middle can adjust the pose, and the URDF on the right will also display the configured pose. After the setup, click 【Save】 to complete the configuration.

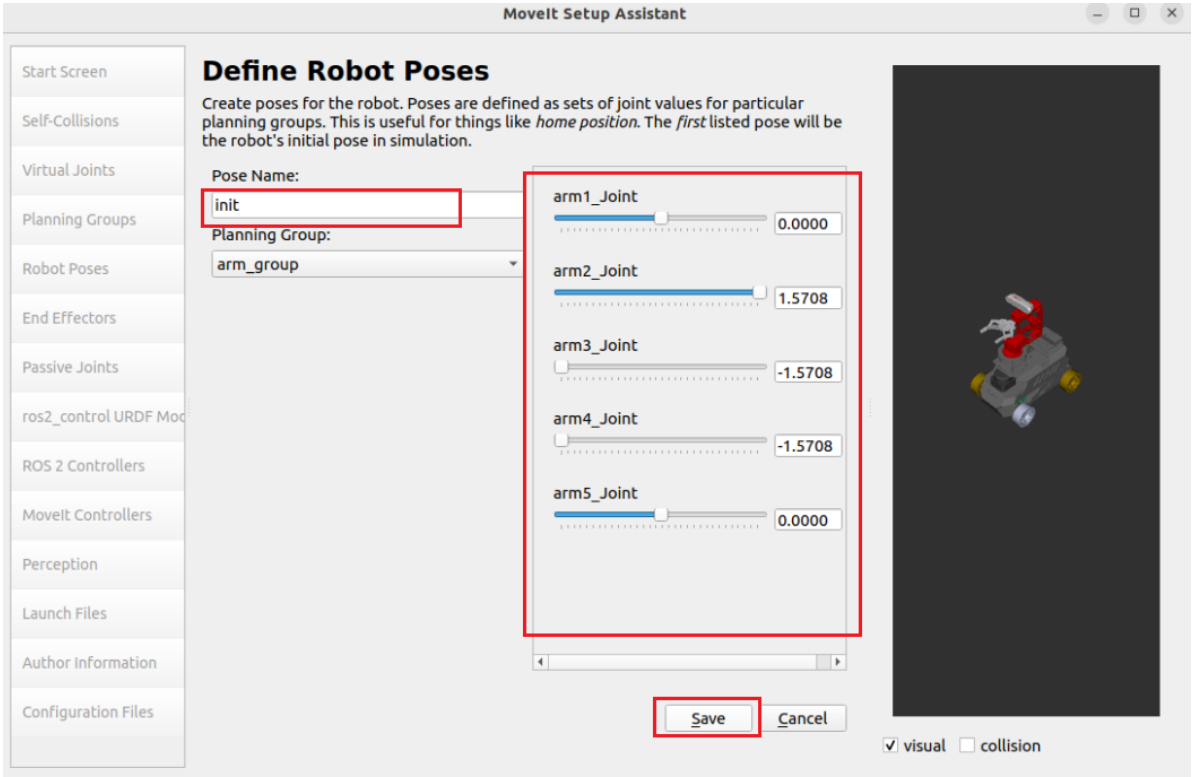


Next, following the same steps used to set the up pose, configure the down pose and init pose of the arm_group planning group. Refer to the figures below:

down pose:

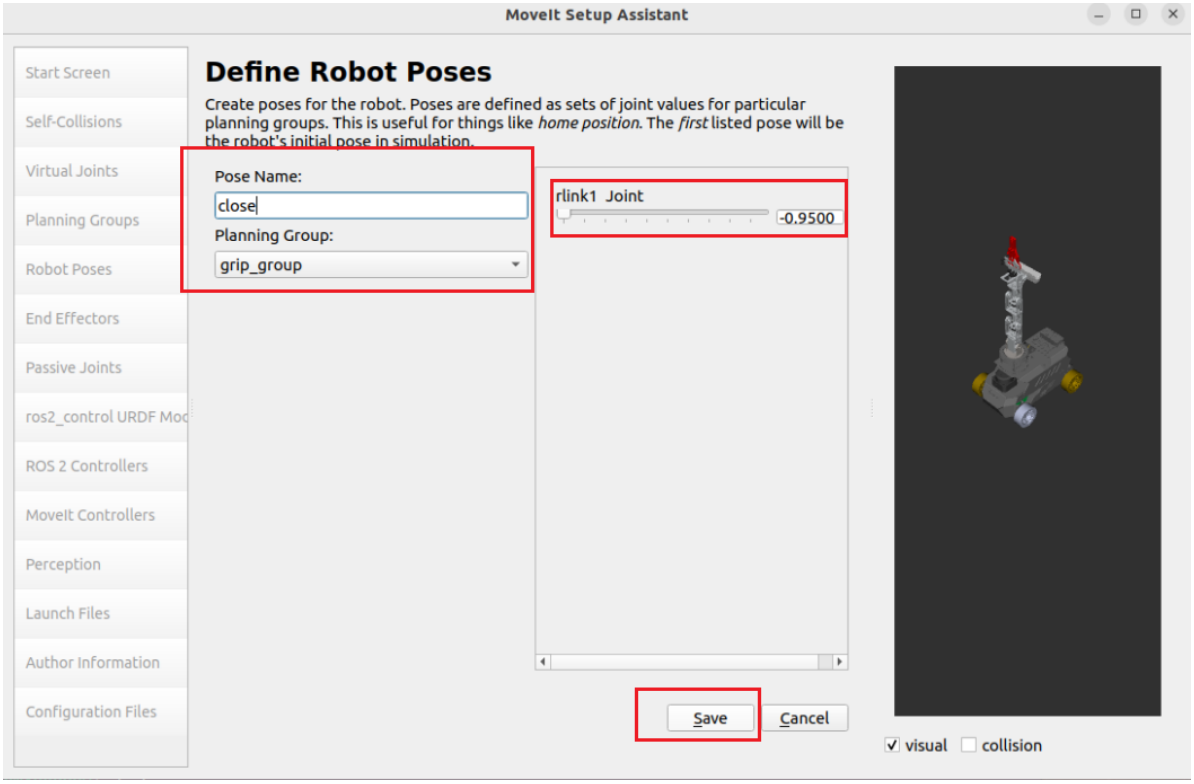


init pose:

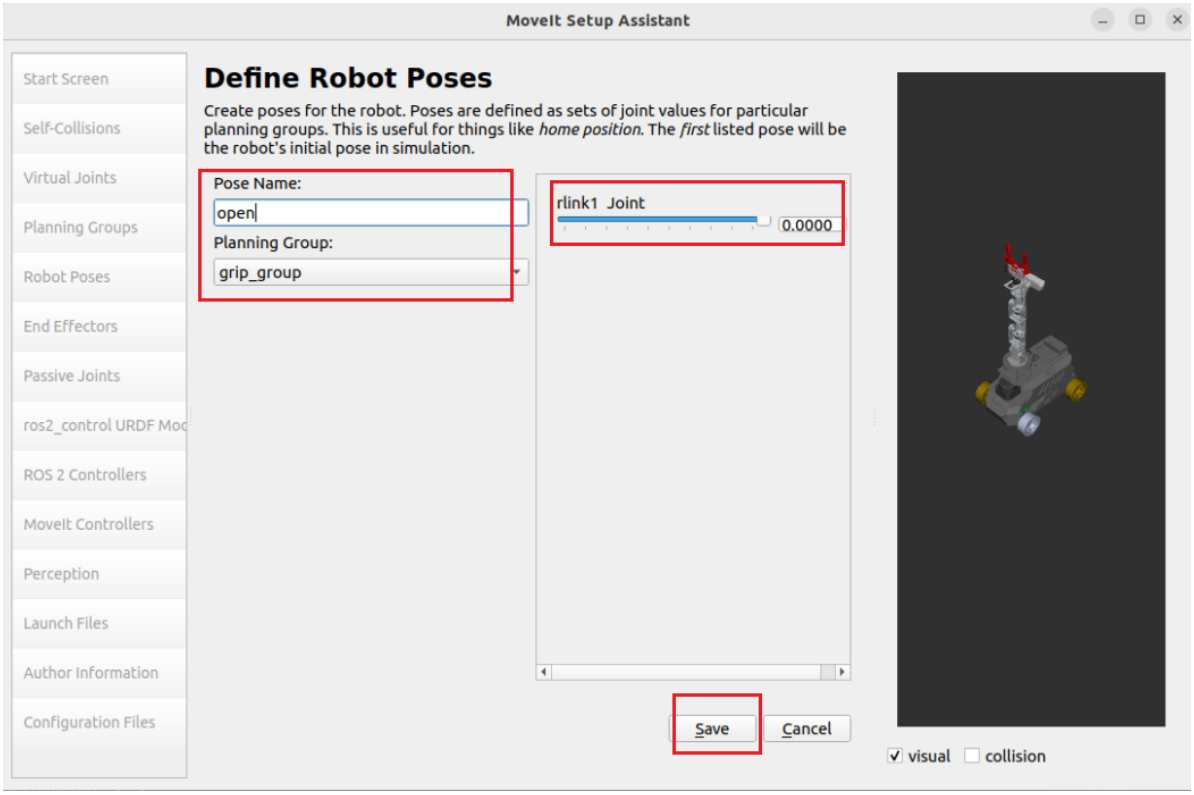


After configuring the three poses of the arm_group group, configure the two poses of the grip_group group, namely close and open. Note that the 【Planning Group】 here needs to be changed to grip_group. Configure it as shown below:

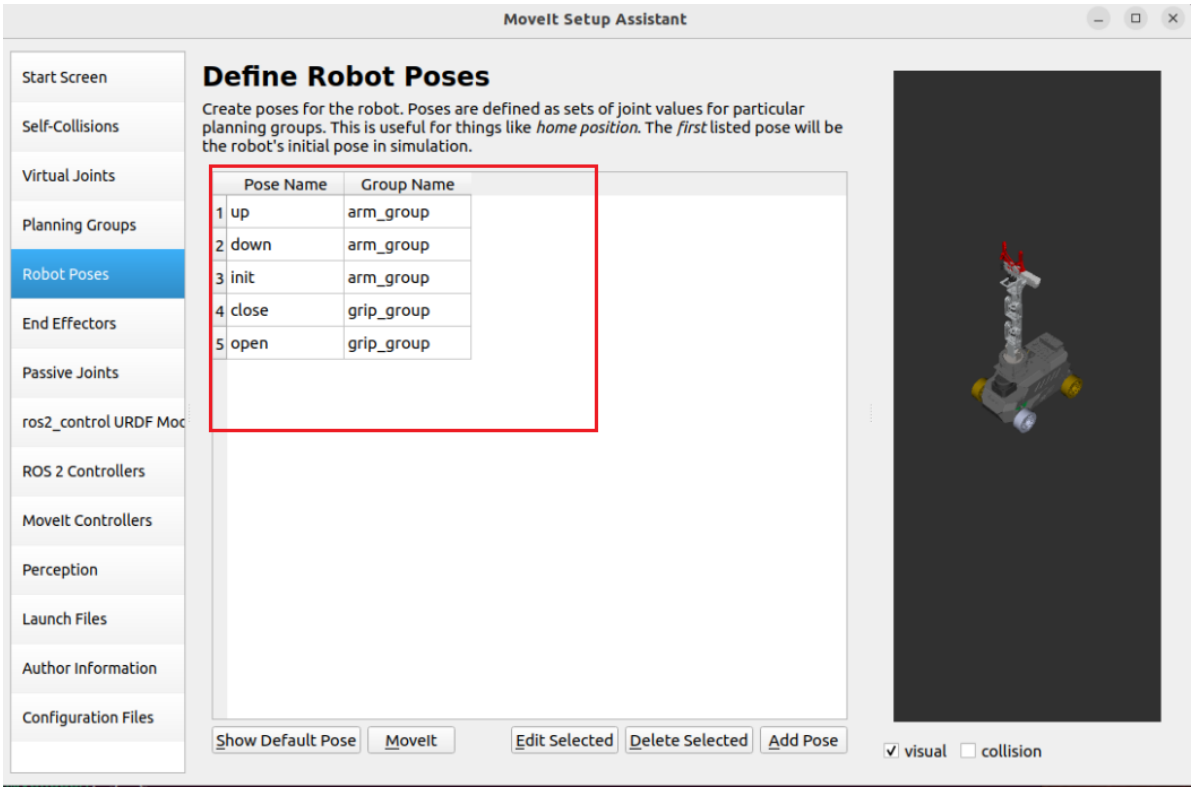
close pose:



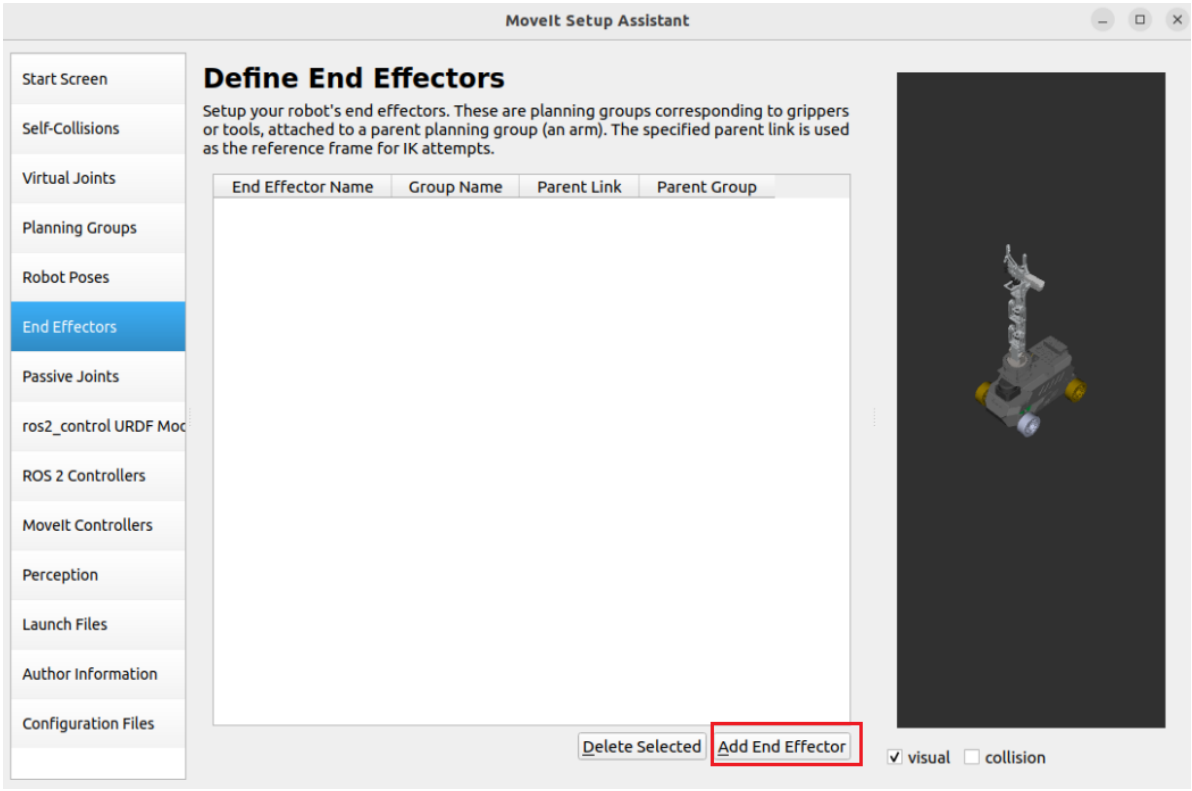
open pose:



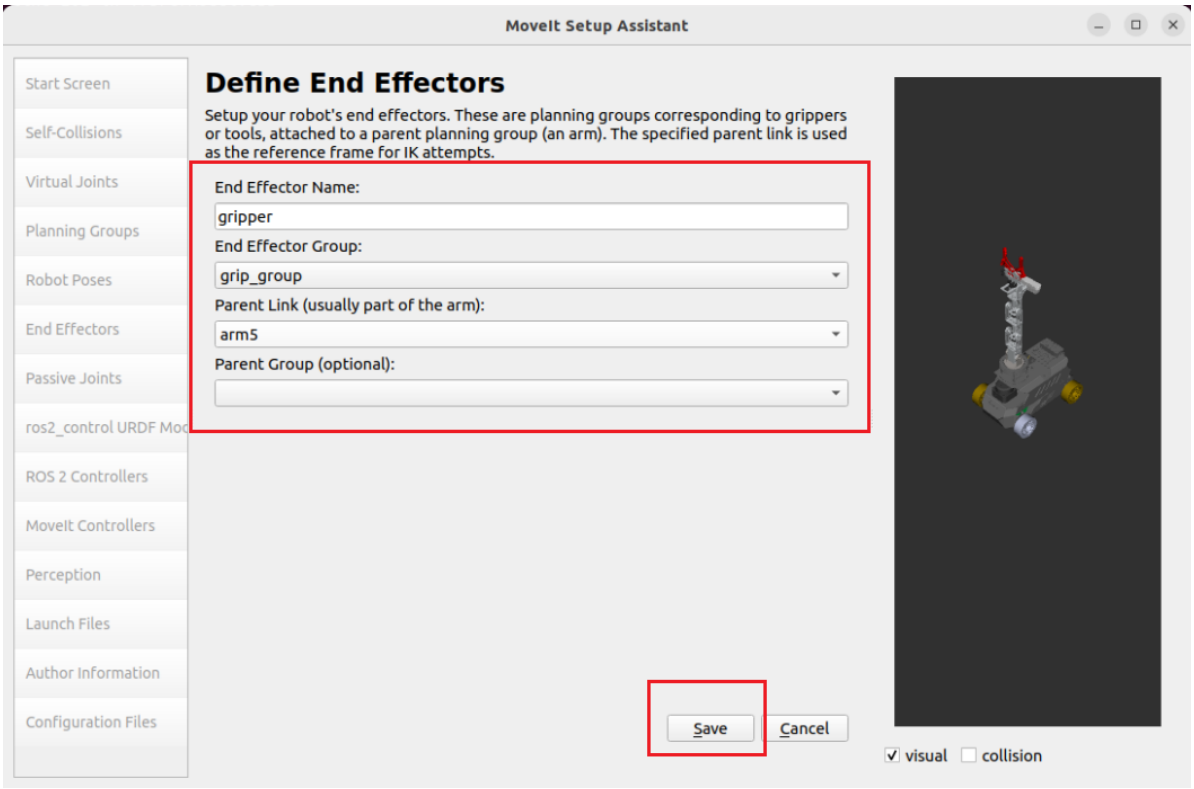
After the 【Robot Poses】 setup is complete, as shown in the figure below:



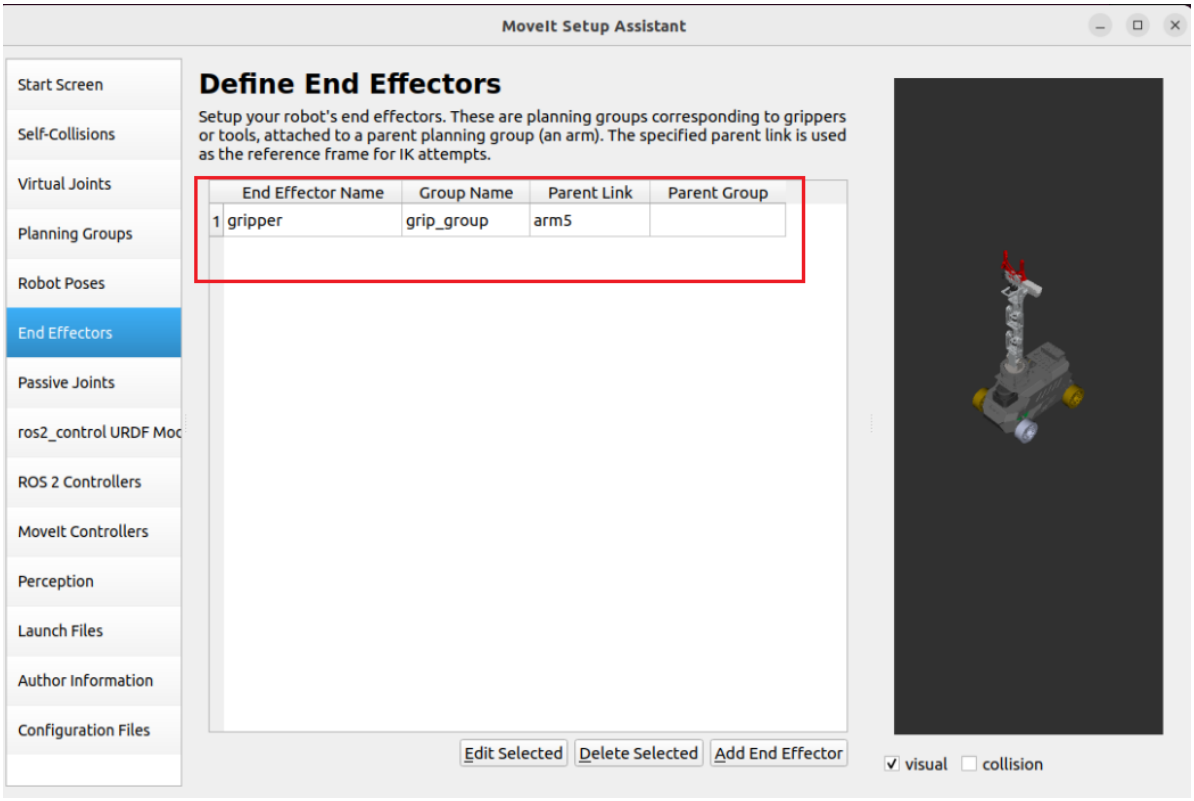
Next, click 【End Effectors】 in the settings bar on the left to configure the end effector. As shown in the figure below, click 【Add End Effector】 to enter the setup interface:



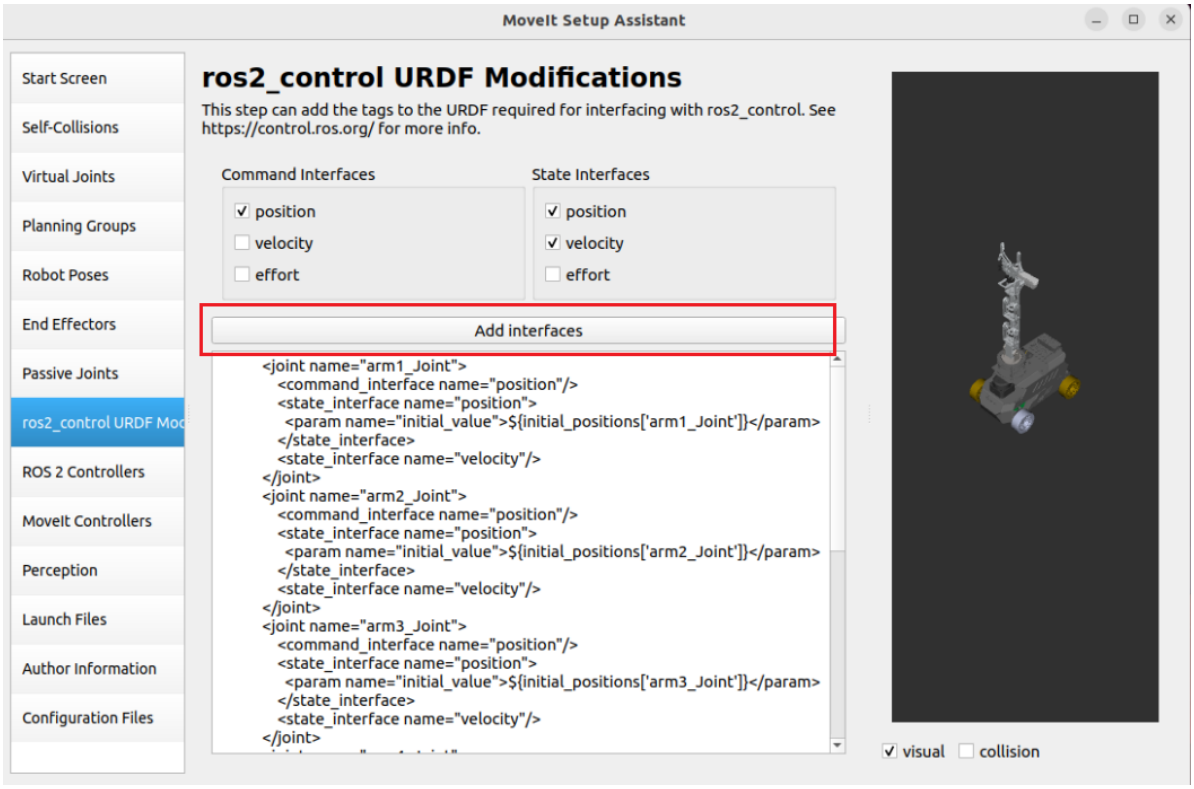
In the setup interface, configure it according to the figure below. After the setup, click 【Save】 to save and exit:



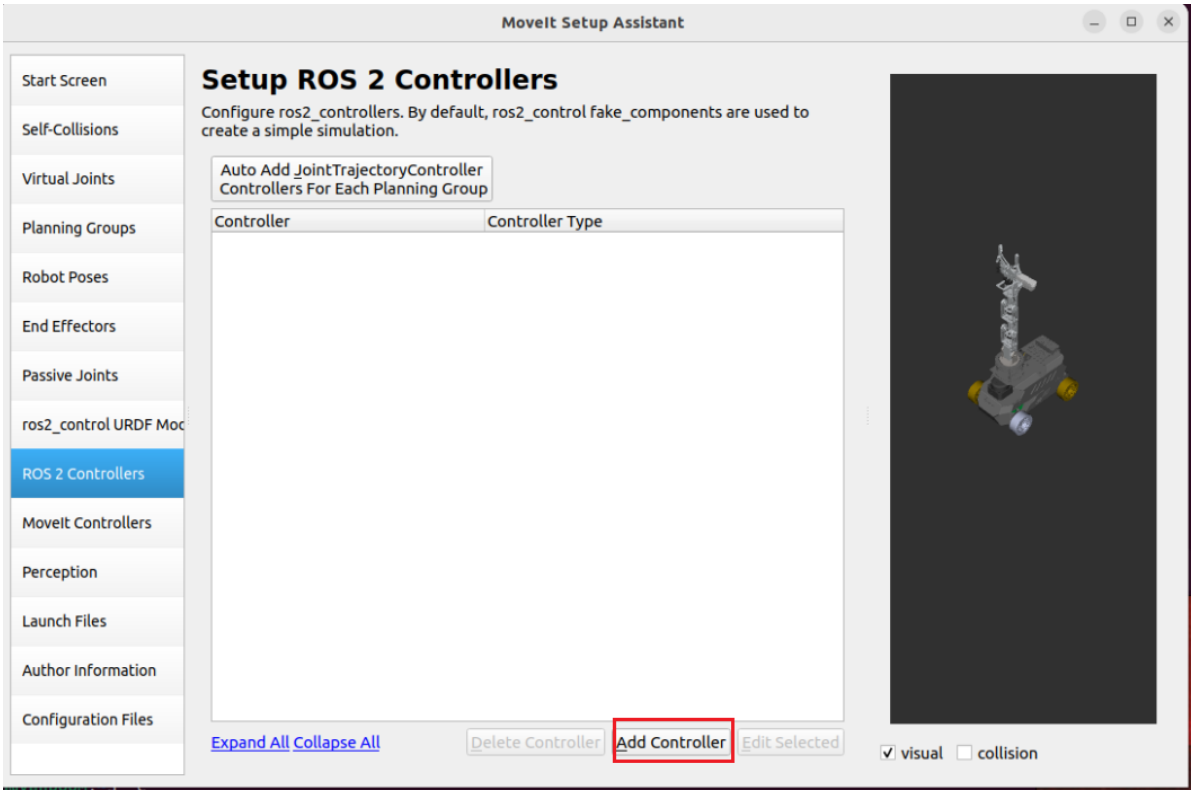
The completed 【End Effectors】 setup is shown in the figure below:



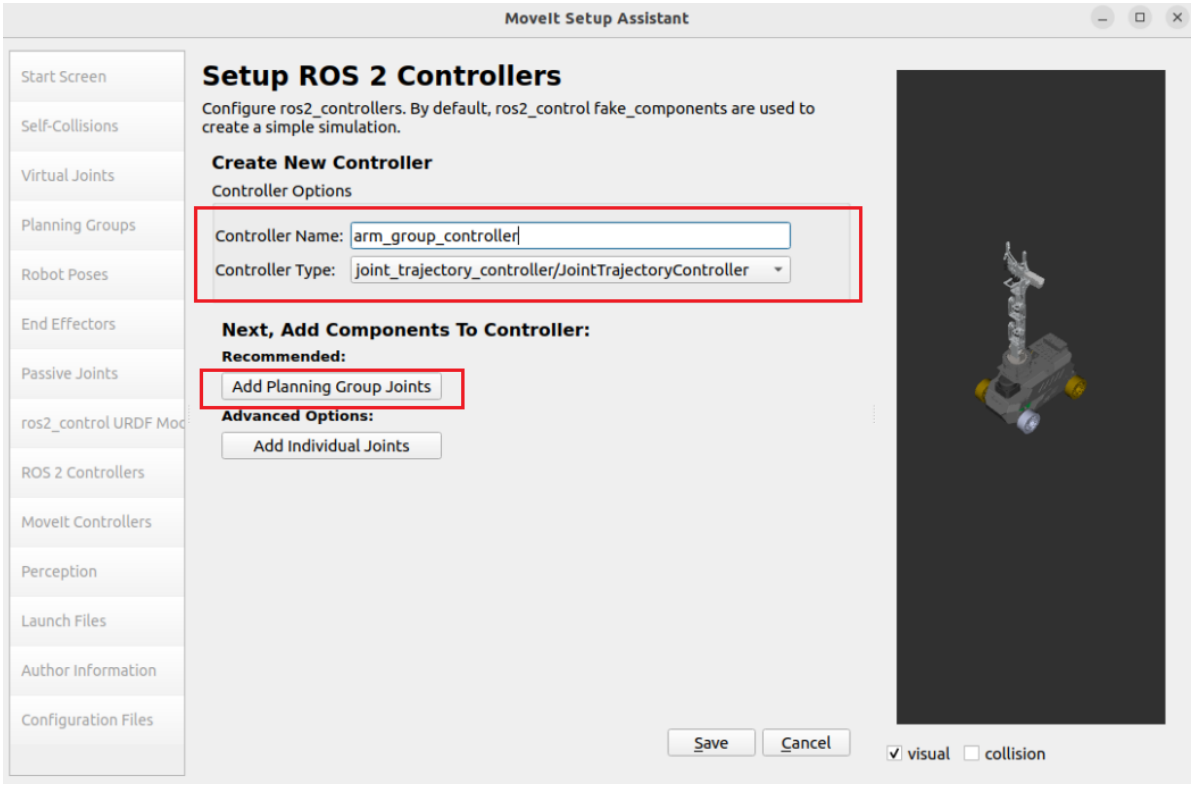
Next, click 【ros2_control URDF Modifications】 in the settings bar on the left. This step is to configure the URDF model. We do not need to modify the URDF model; simply click 【Add interfaces】, as shown in the figure below:



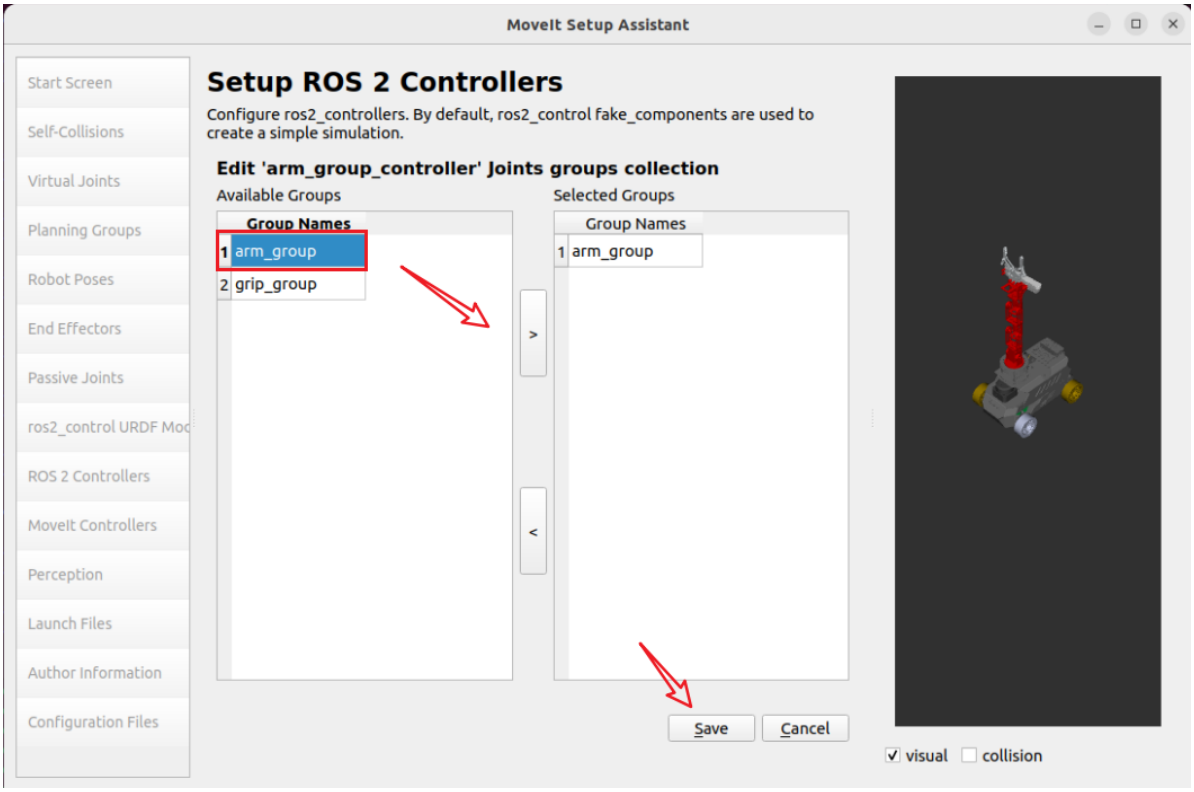
Click 【ROS2 Controllers】 in the settings bar on the left. This step is to configure robot motion control. Click 【Add Controller】 to add a controller, as shown in the figure below:



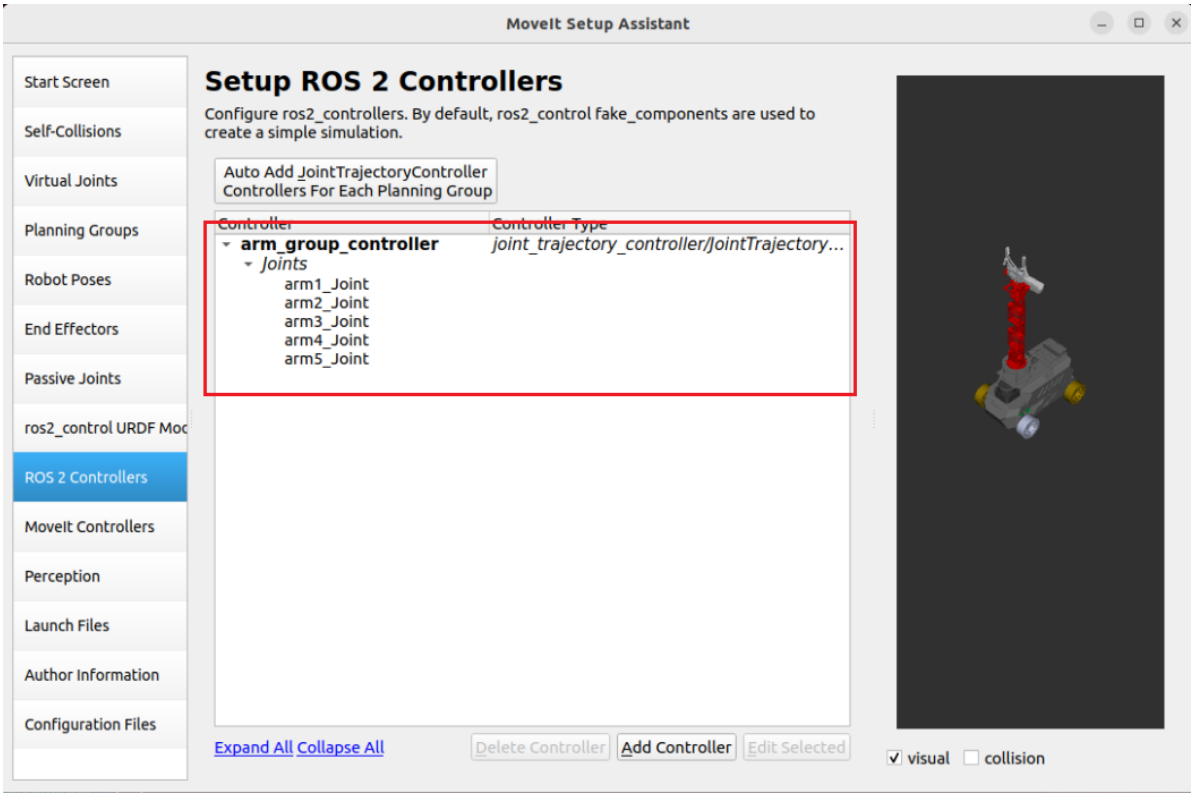
Once inside the settings, follow the figure below to add the arm_group_controller:



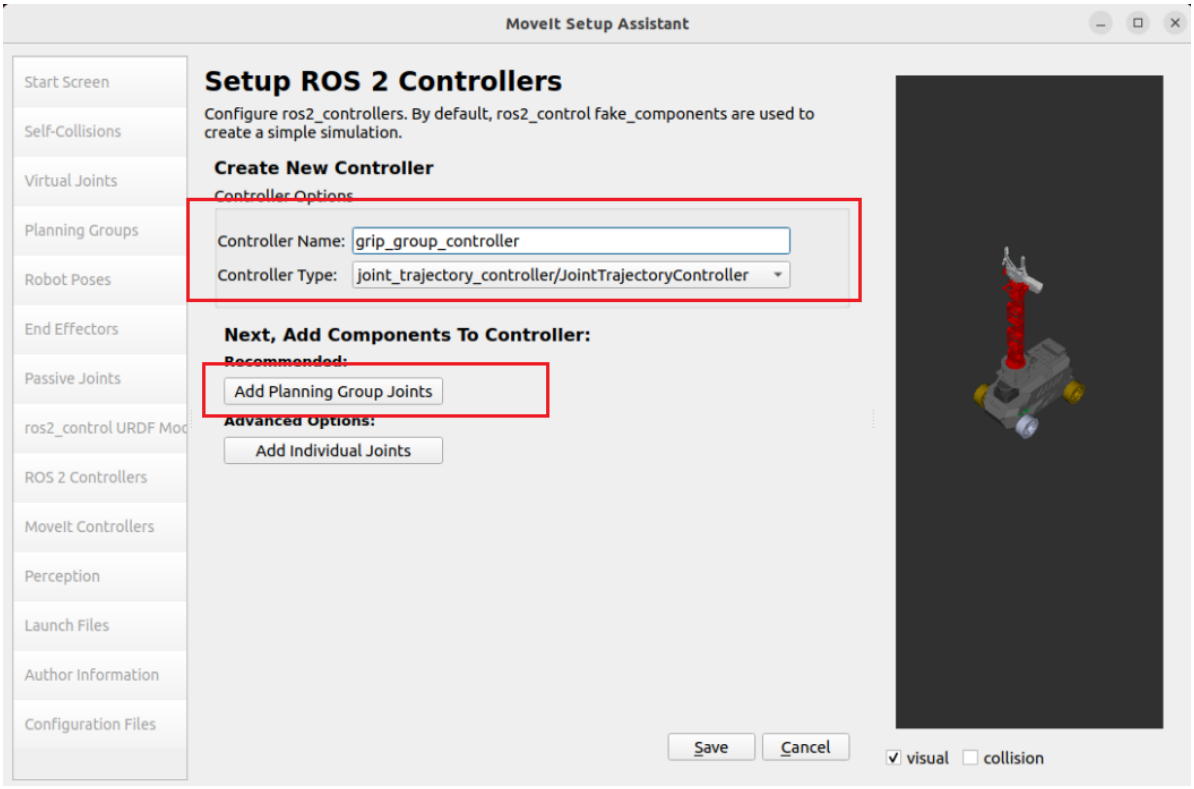
Then click 【Add Planning Group Joints】 to add the joints of the control group. As shown in the figure below, select arm_group on the left and click > to complete the addition. Finally, click 【Save】 to save and exit:



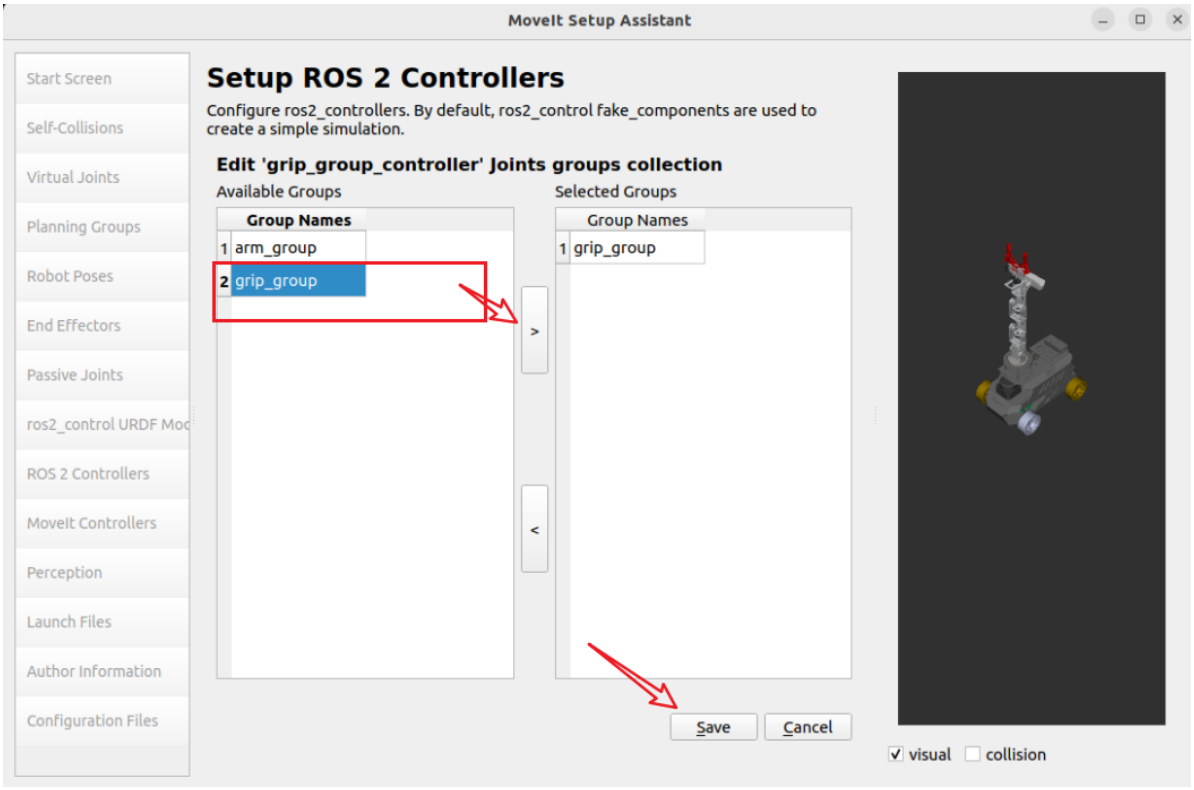
The first controller after setup is shown in the figure below:



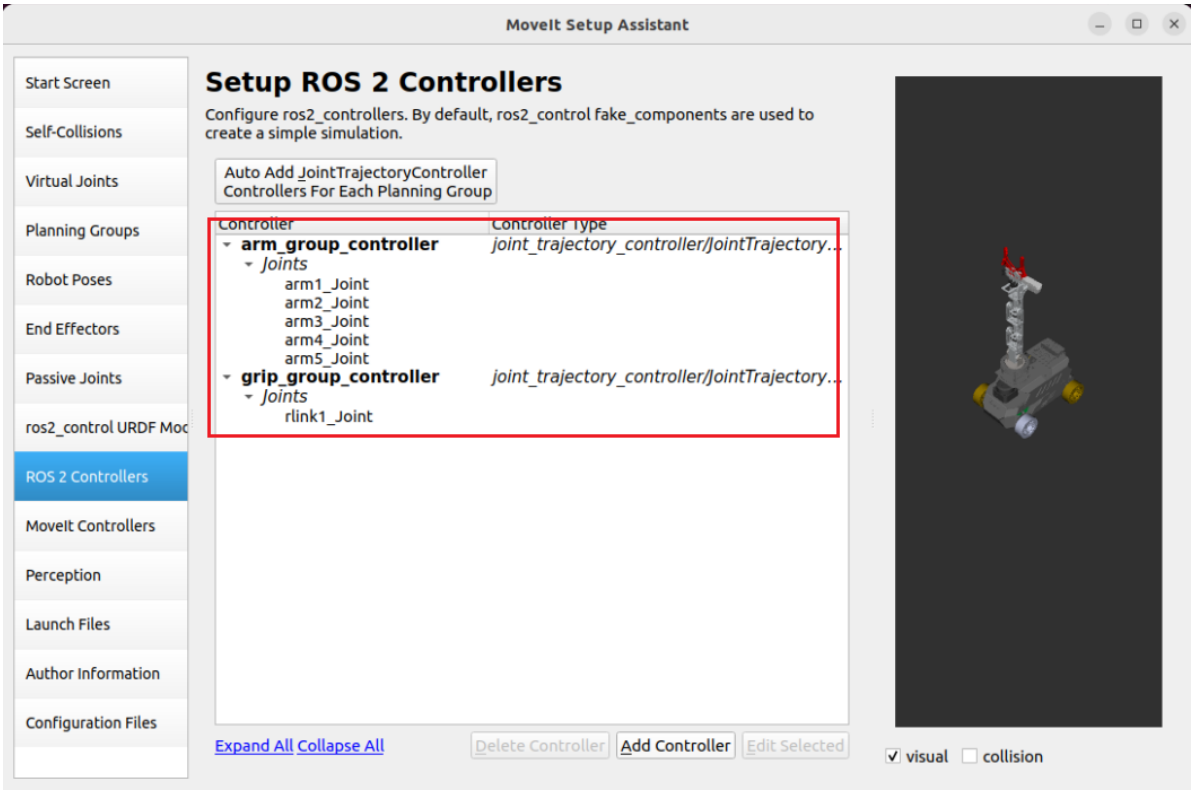
Then we add the second controller. Again, click 【Add Controller】 to enter the add interface, and configure it according to the figure below:



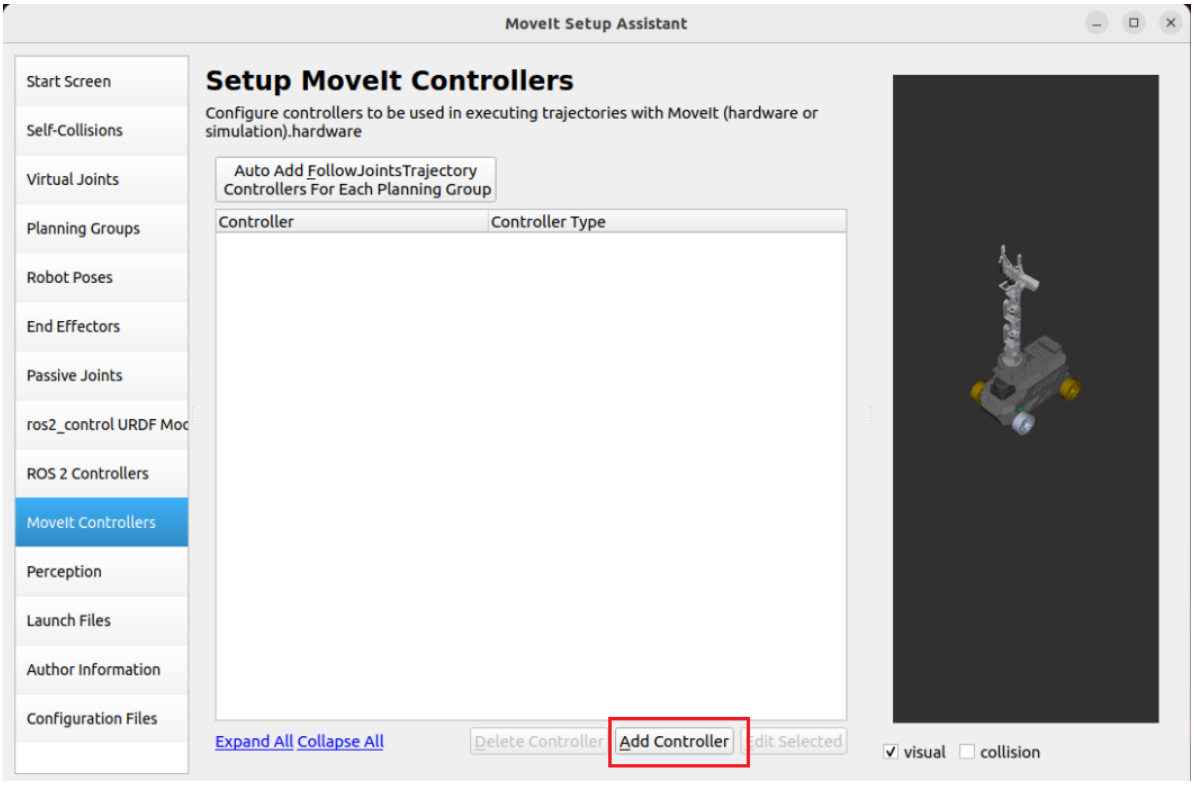
Then click 【Add Planning Group Joints】 to add joints. As shown in the figure below, select grip_group on the left and click > to complete the addition. Finally, click 【Save】 to complete the configuration:



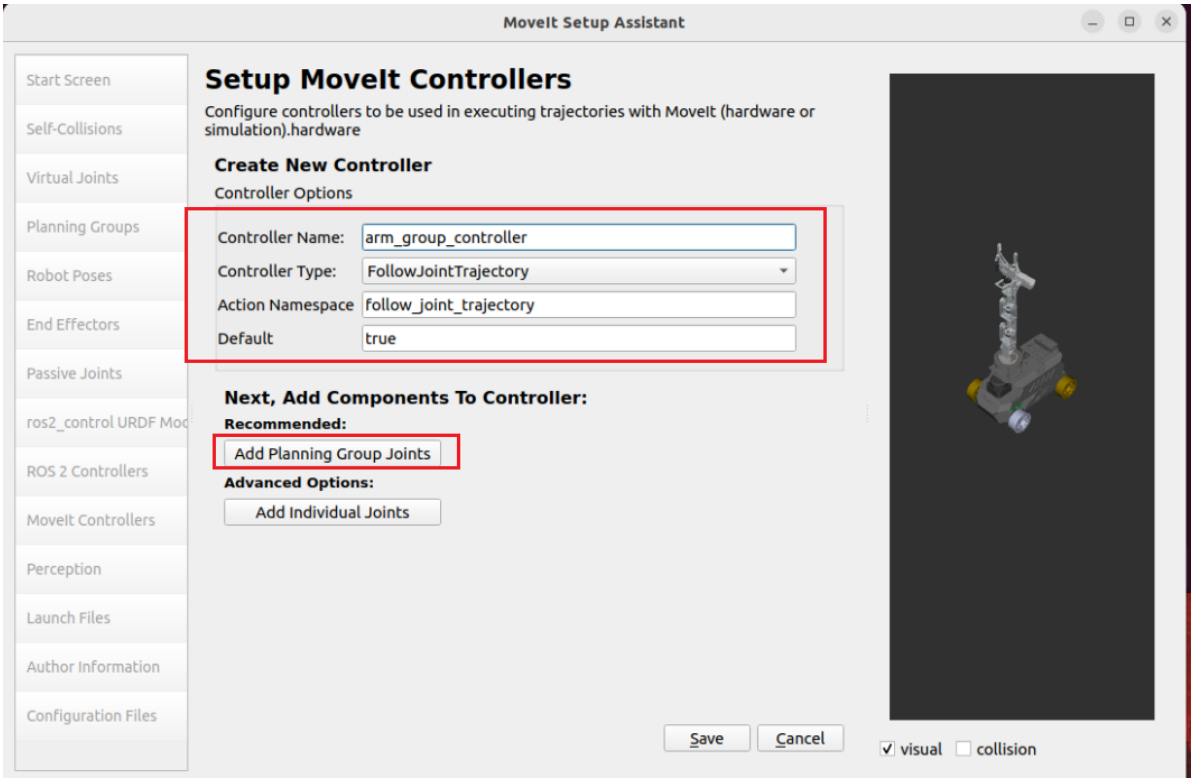
After the 【ROS2 Controllers】 setup is complete, as shown in the figure below:



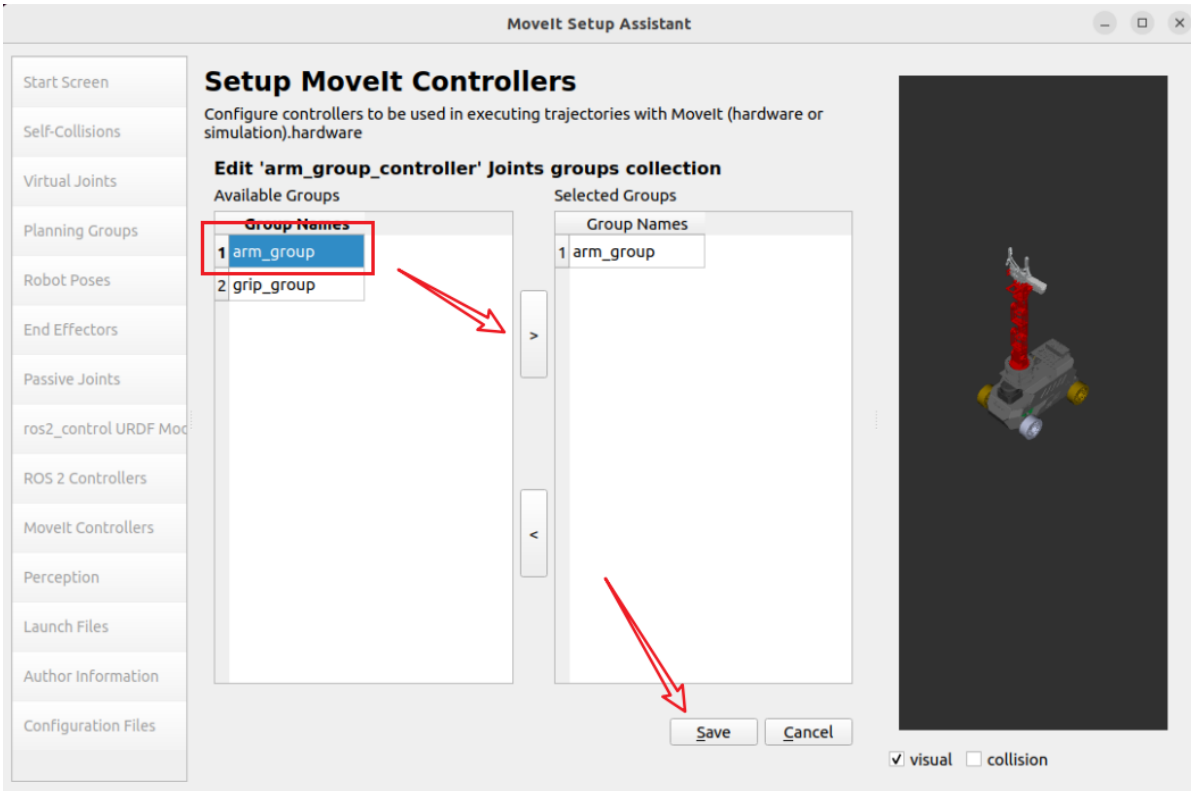
Next, click 【MoveIt Controllers】 in the settings bar on the left. This step is to configure the trajectory controller to execute the planned trajectories and send the generated trajectories to the robot's ROS2 controller. Click 【Add Controller】 to add a trajectory controller, as shown in the figure below:



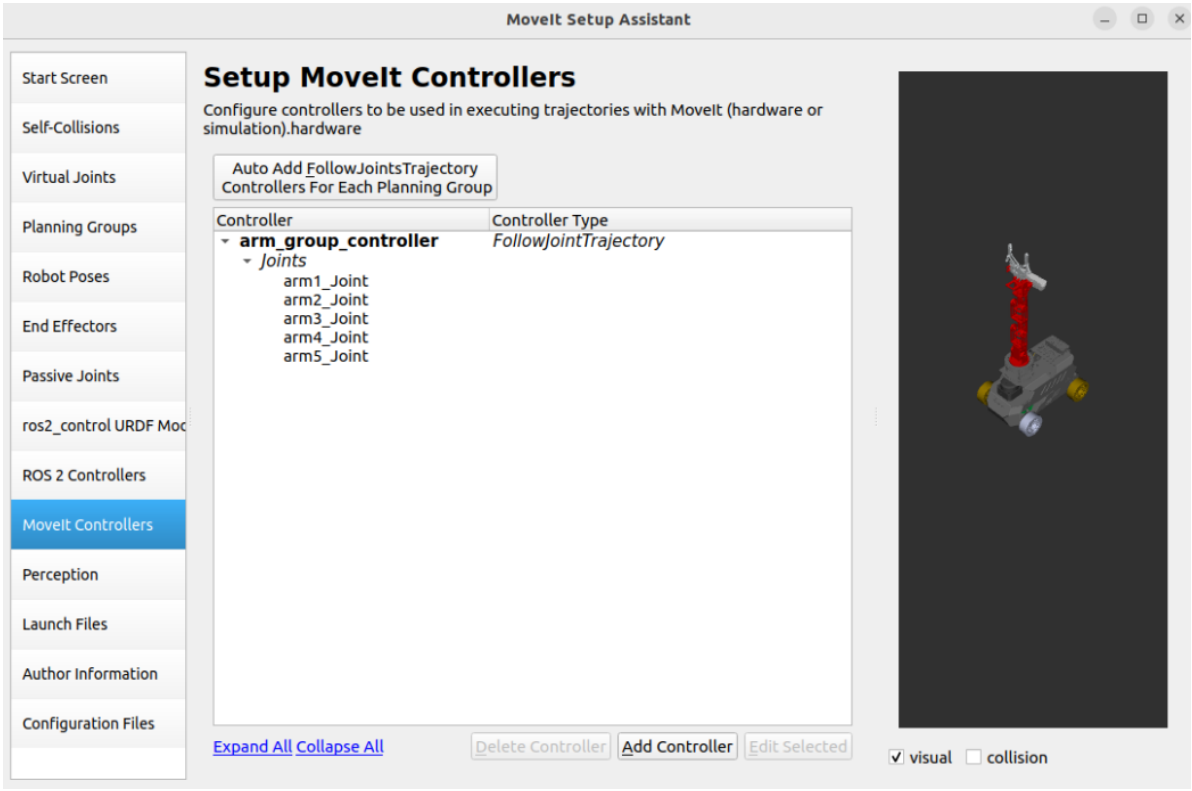
Once inside the setup interface, configure it according to the figure below. Note that the 【Controller Name】 here must match the 【ROS2 Controller】 set in the previous step, and then click 【Add Planning Group Joints】 to add the control joints:



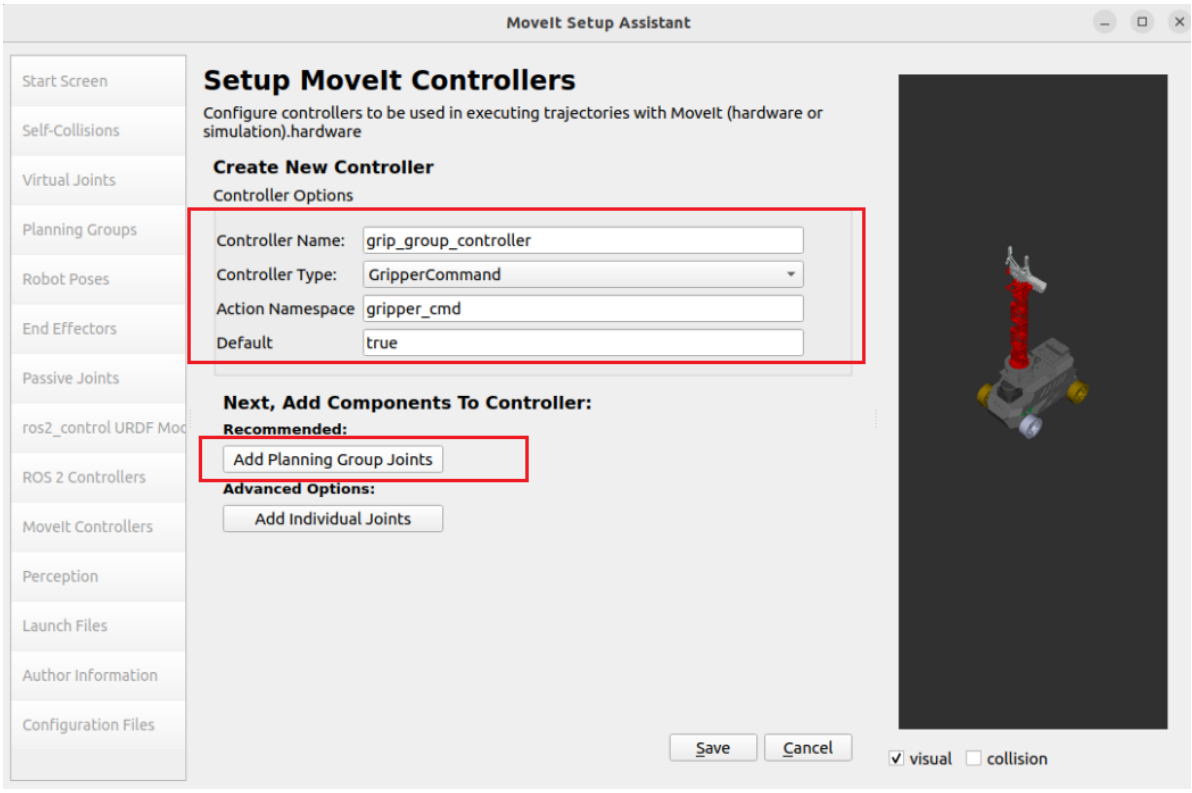
As shown in the figure below, add the arm_group control group. Select arm_group on the left and click > to complete the addition. Finally, click 【Save】 to save and exit:



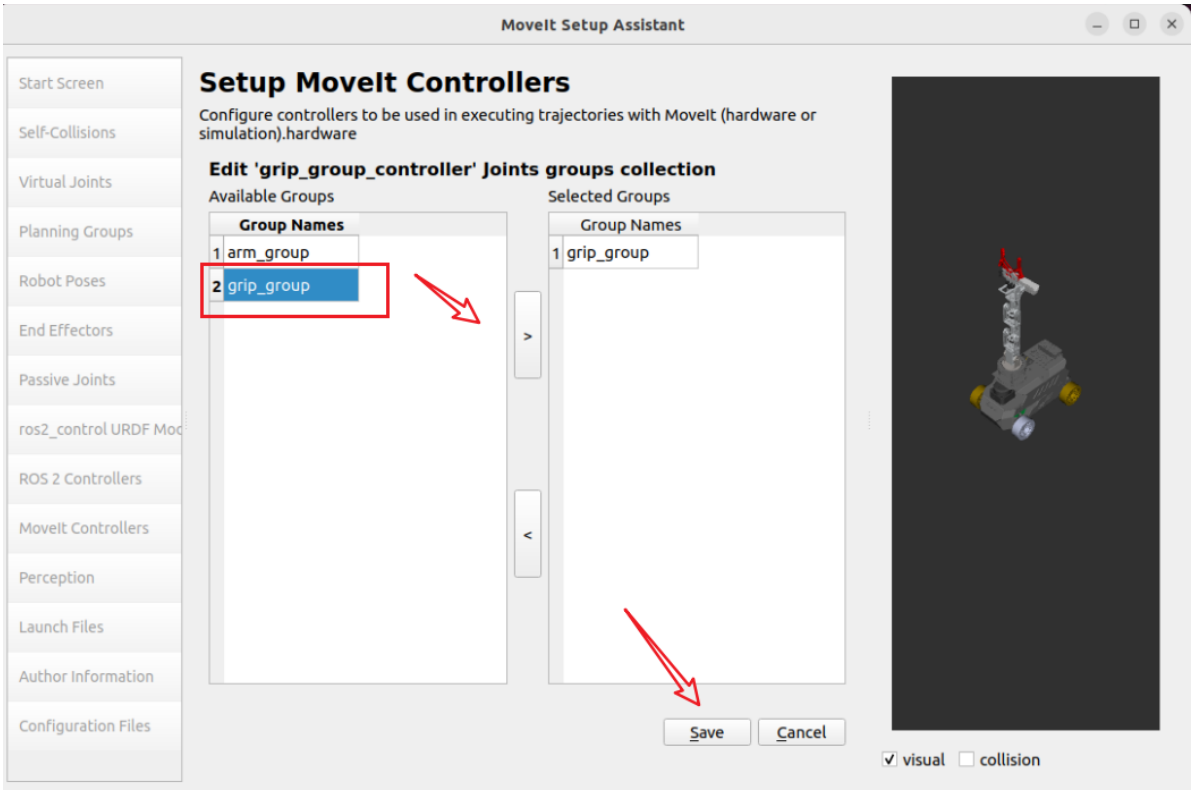
After the setup is complete, as shown in the figure below:



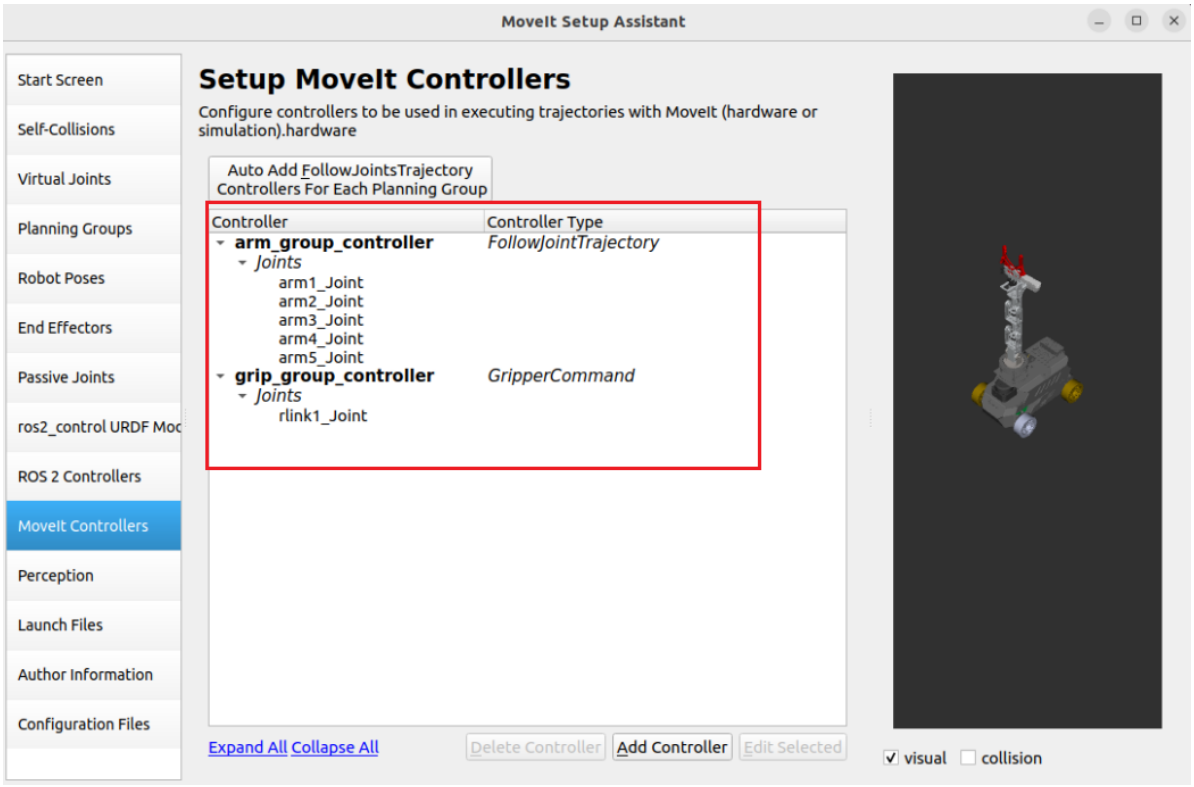
Next, add the trajectory controller for the gripper. Click 【Add Controller】 to enter the setup interface, configure it according to the figure below, and then click 【Add Planning Group Joints】 to add the control joints:



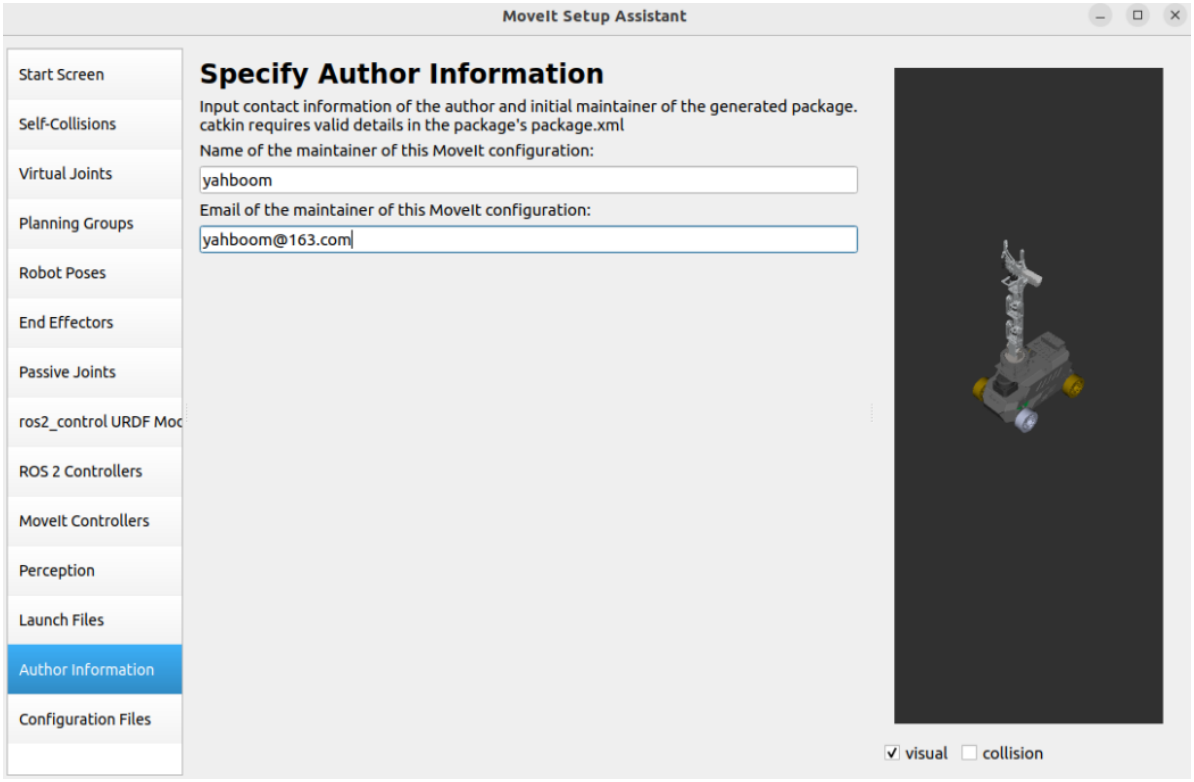
Then click 【Add Planning Group Joints】 to add joints. As shown in the figure below, select grip_group on the left and click > to complete the addition. Finally, click 【Save】 to complete the configuration:



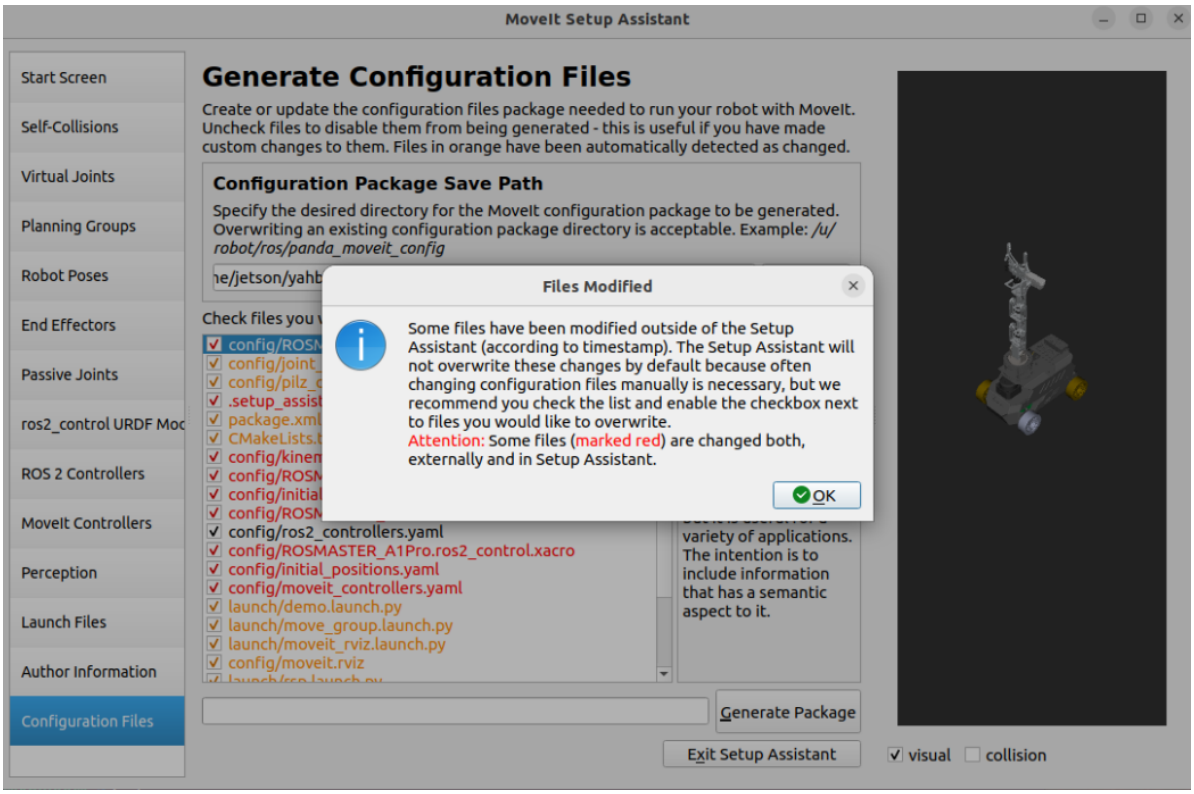
The completed 【MoveIt Controllers】 setup is shown in the figure below:



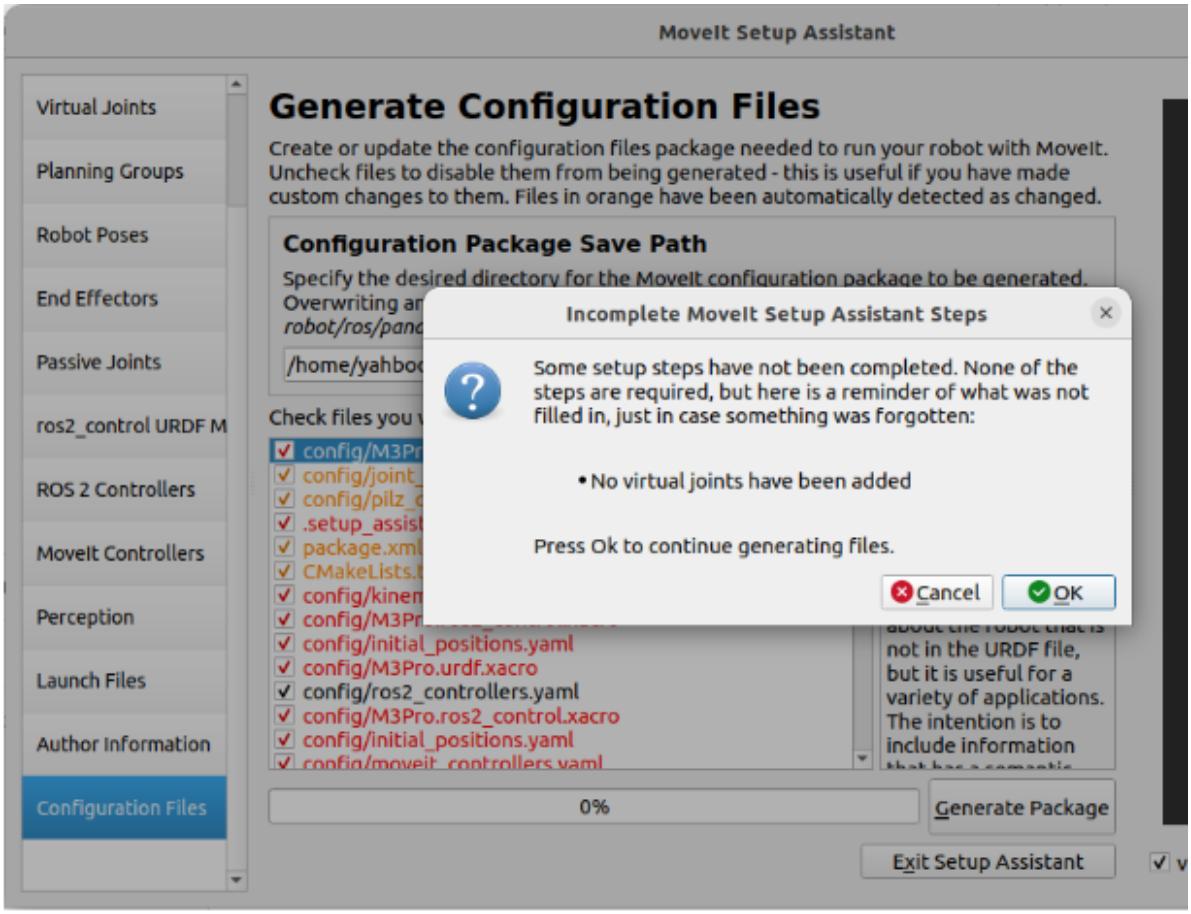
Next, click 【Author Information】 in the settings bar on the left, then fill in your name and email address, as shown in the figure below:



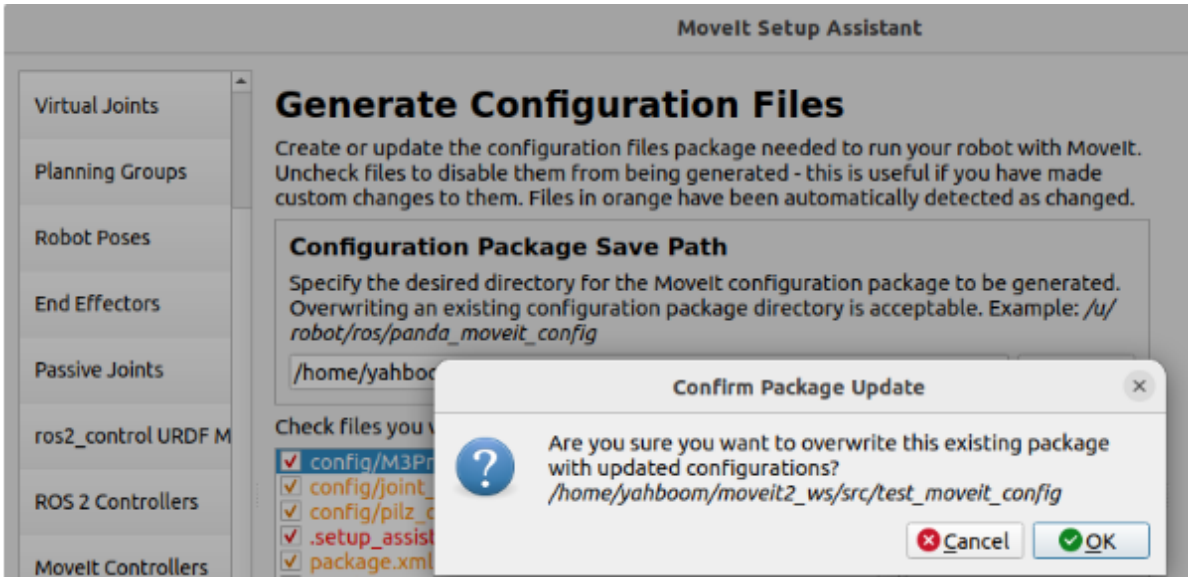
Finally, click 【Configuration Files】 in the settings bar on the left, click 【Browse】 to find the folder we created earlier. Here `/home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_moveit` is selected. When the following information appears, click 【ok】:



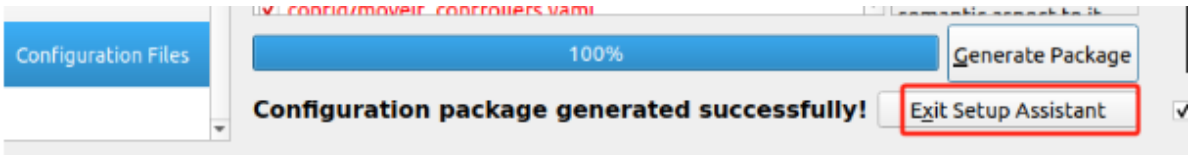
Then click 【Generate Package】 below to generate the package. As shown in the figure below, when a prompt appears, click 【ok】:



When a prompt appears, click 【ok】:



As shown in the figure below, when **Configuration package generated successfully** appears, click 【Exit Setup Assistant】 to complete all the setup:



Everything configured just now is saved in the test_moveit_config package. Next, we need to modify `/home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_moveit/config/joint_limits.yaml` — simply change the integers inside to decimals while keeping the values unchanged. The modified file is as follows:

```yaml
# joint_limits.yaml allows the dynamics properties specified in the URDF to be
overwritten or augmented as needed

# For beginners, we downscale velocity and acceleration limits.
# You can always specify higher scaling factors (<= 1.0) in your motion requests.
# Increase the values below to 1.0 to always move at maximum speed.
default_velocity_scaling_factor: 0.1
default_acceleration_scaling_factor: 0.1

# Specific joint properties can be changed with the keys [max_position,
min_position, max_velocity, max_acceleration]
# Joint limits can be turned off with [has_velocity_limits,
has_acceleration_limits]
joint_limits:
  arm1_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  arm2_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  arm3_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  arm4_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  arm5_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  llink1_Joint:
    has_velocity_limits: false
    max_velocity: 0.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  llink2_Joint:
    has_velocity_limits: false
    max_velocity: 0.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  llink3_Joint:
    has_velocity_limits: false
    max_velocity: 0.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  rlink1_Joint:
    has_velocity_limits: true
    max_velocity: 1.0
```

```
    has_acceleration_limits: false
    max_acceleration: 0.0
  rlink2_Joint:
    has_velocity_limits: false
    max_velocity: 0.0
    has_acceleration_limits: false
    max_acceleration: 0.0
  rlink3_Joint:
    has_velocity_limits: false
    max_velocity: 0.0
    has_acceleration_limits: false
    max_acceleration: 0.0
```

Save and exit, then return to the workspace directory and use colcon build to compile. Enter the following in the terminal:

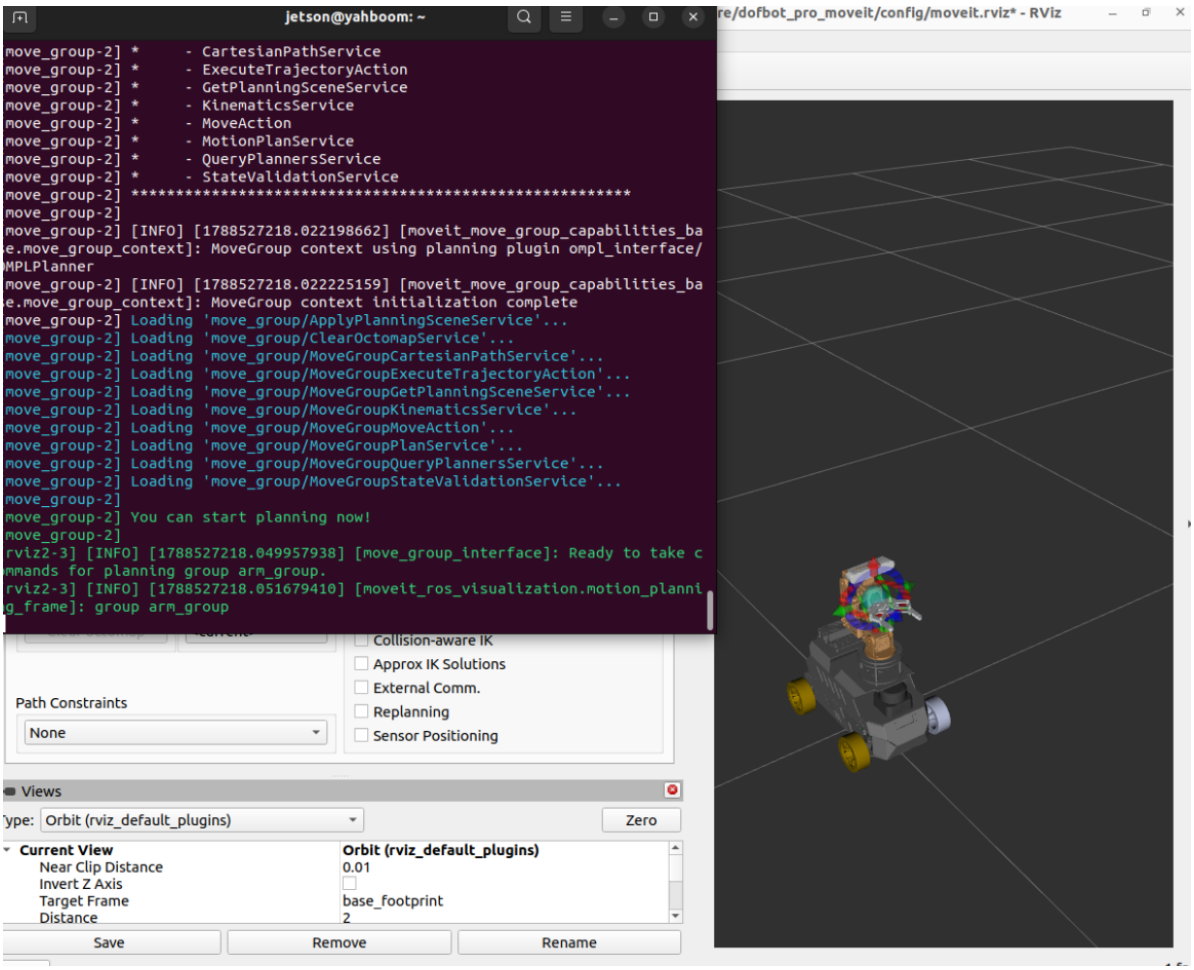
```
cd ~/yahboomcar_ros2_ws/dofbot_pro_ws
colcon build --packages-select dofbot_pro_moveit
```

After compilation, refresh the environment variables by entering `source ~/.bashrc` in the terminal. Then enter the following command to launch the MoveIt2 simulation environment:

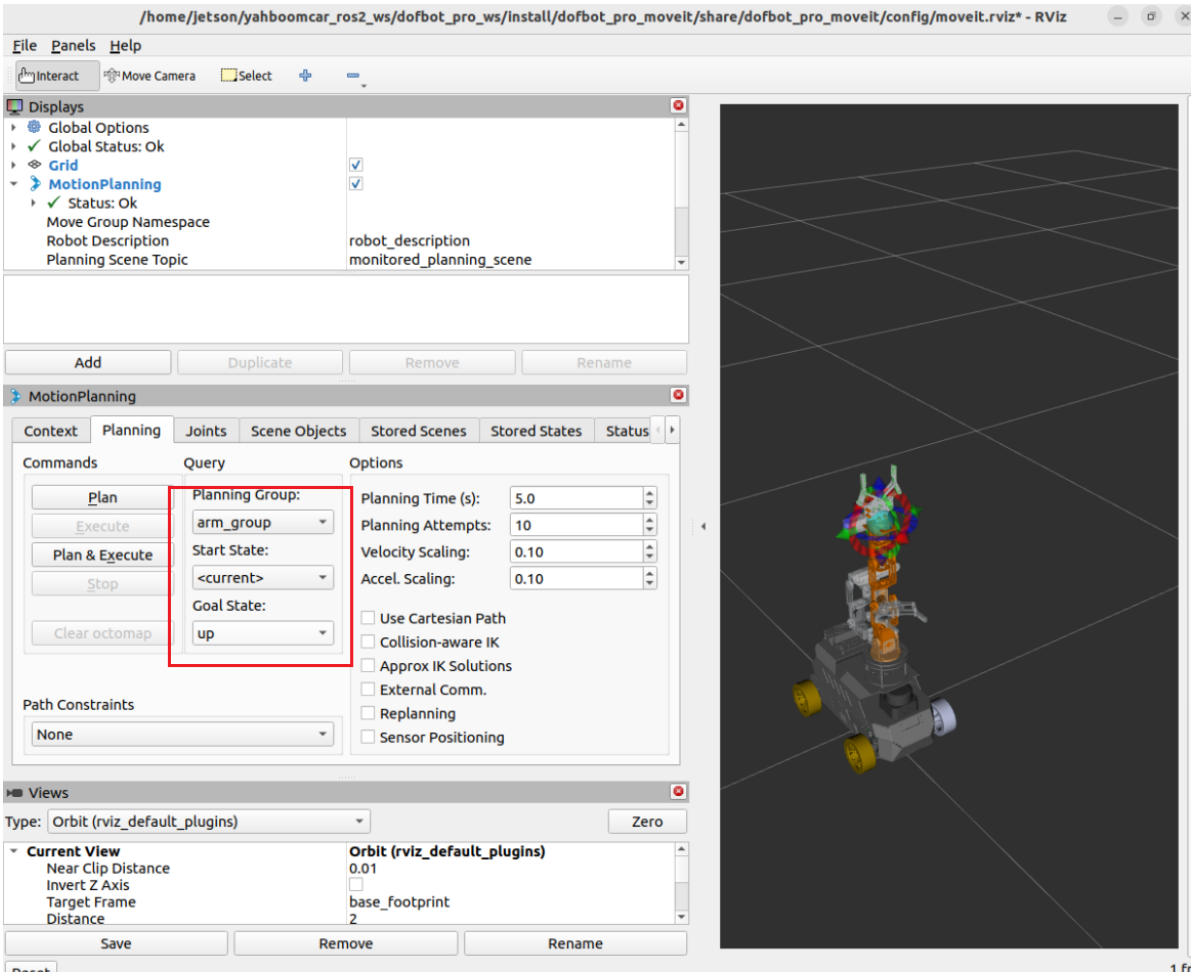
```
ros2 launch dofbot_pro_moveit demo.launch.py
```

> [!NOTE]
> If you saved the configuration package under a custom name (such as `test_moveit_config`) in the Setup Assistant steps above, simply replace the package name accordingly when launching. The car's source code already comes with a dedicated, pre-tuned `dofbot_pro_moveit` package, so it is recommended to use the preset package directly for the subsequent experiments.

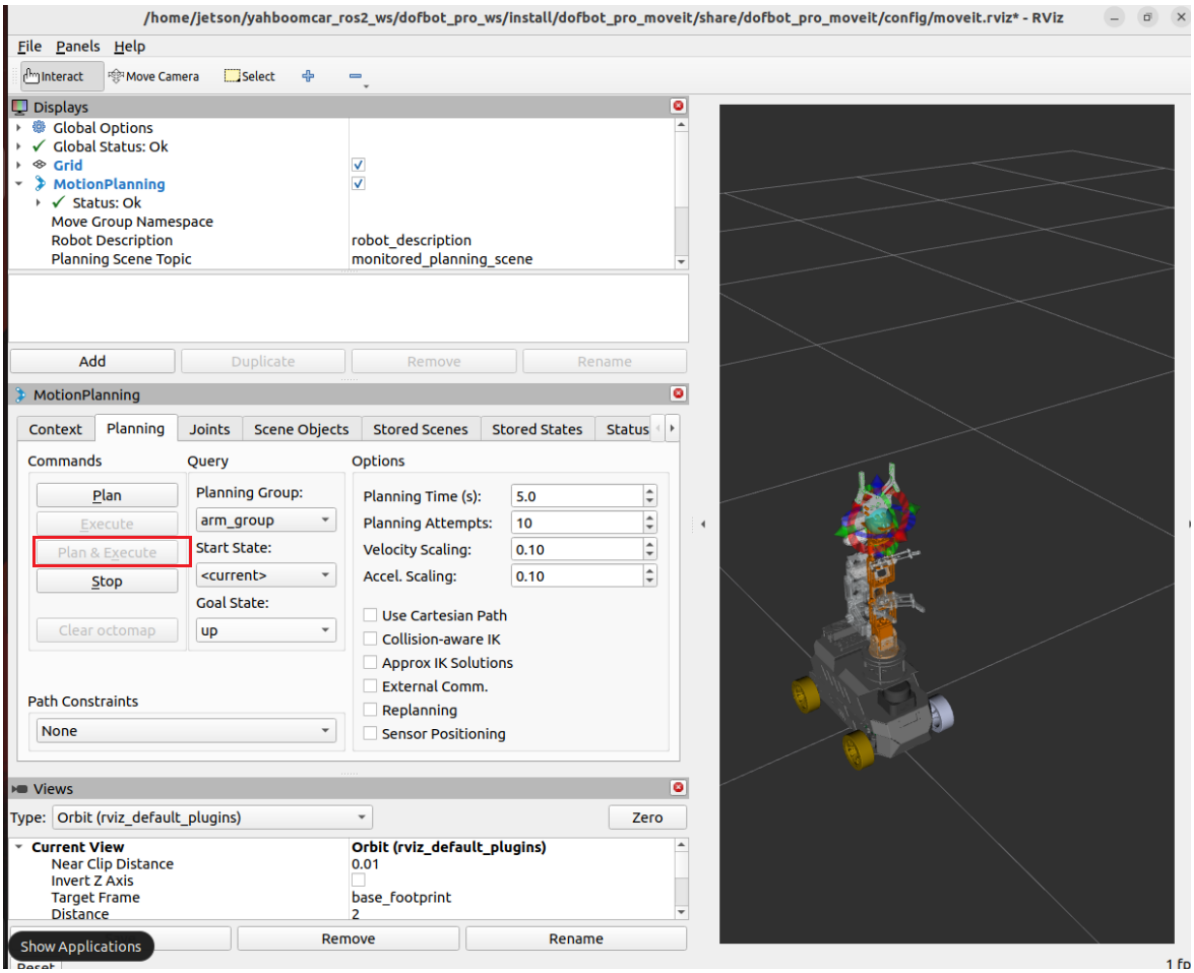
Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



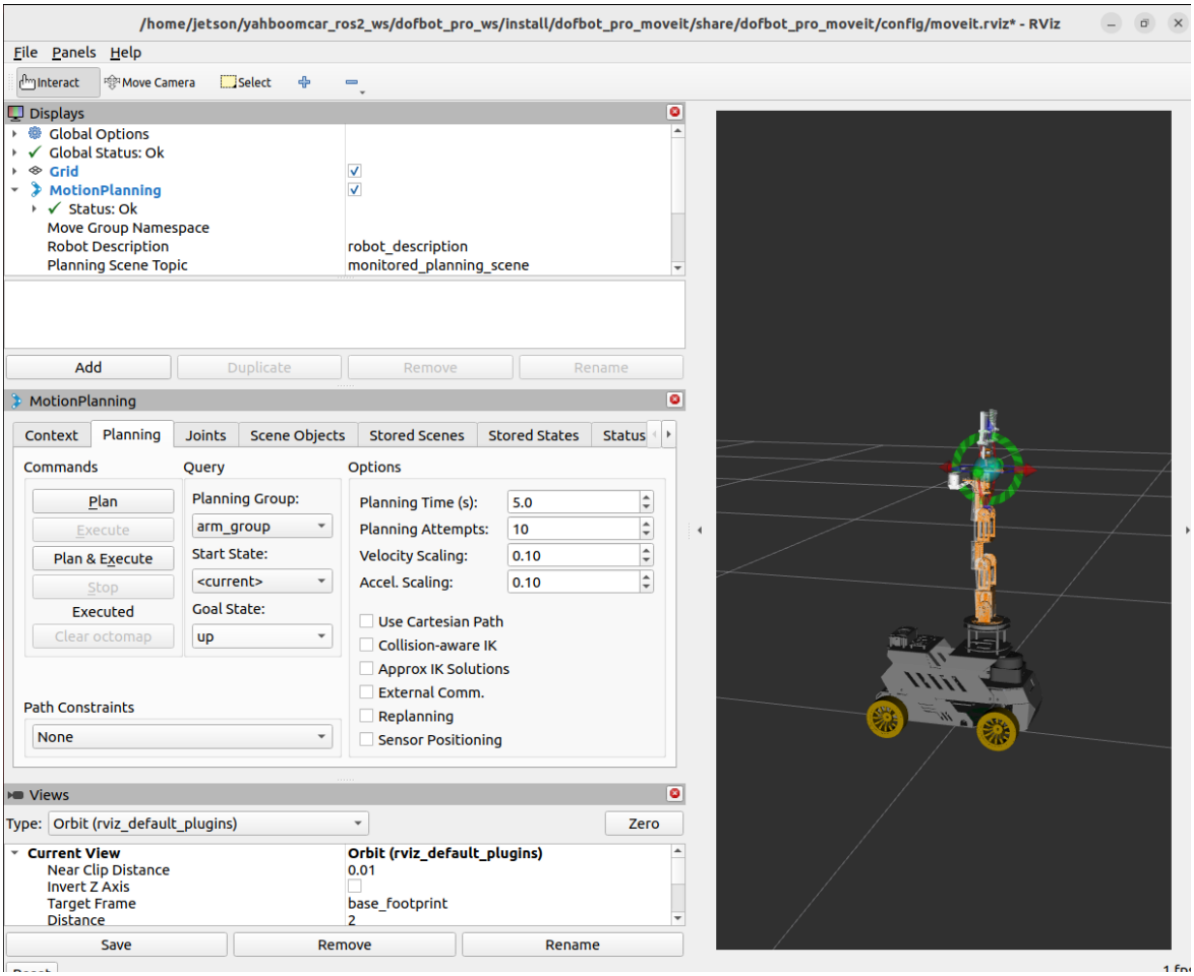
Then let's test Plan and Execute. We first test arm_group. As shown in the figure below, set 【Planning Group】 to arm_group, 【Start State】 to , and 【Goal State】 to . Let the robot arm move from its current init pose to the up pose configured earlier:



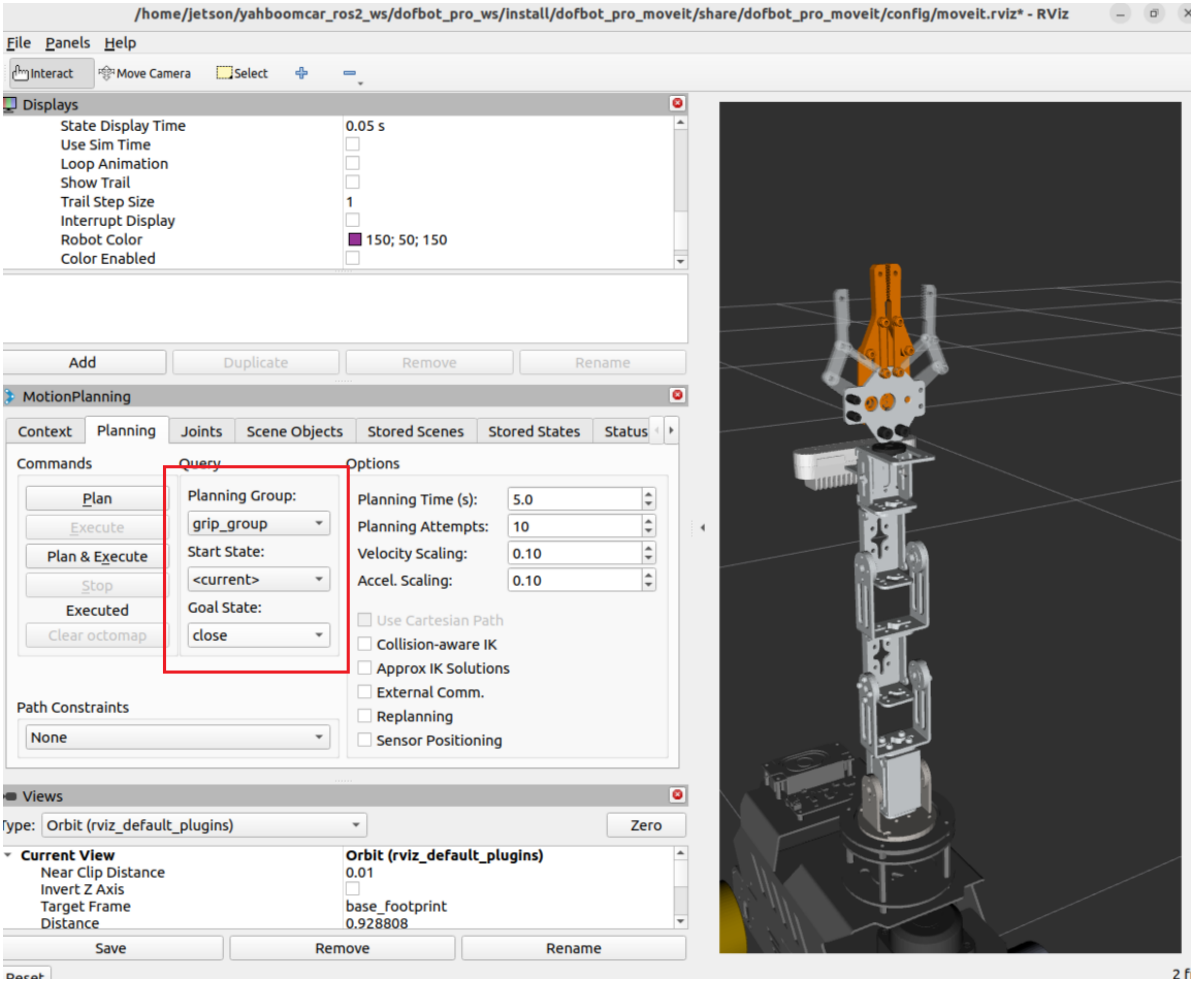
Then click 【Plan & Execute】 on the left to let the robot arm plan and move to init, as shown in the figure below:



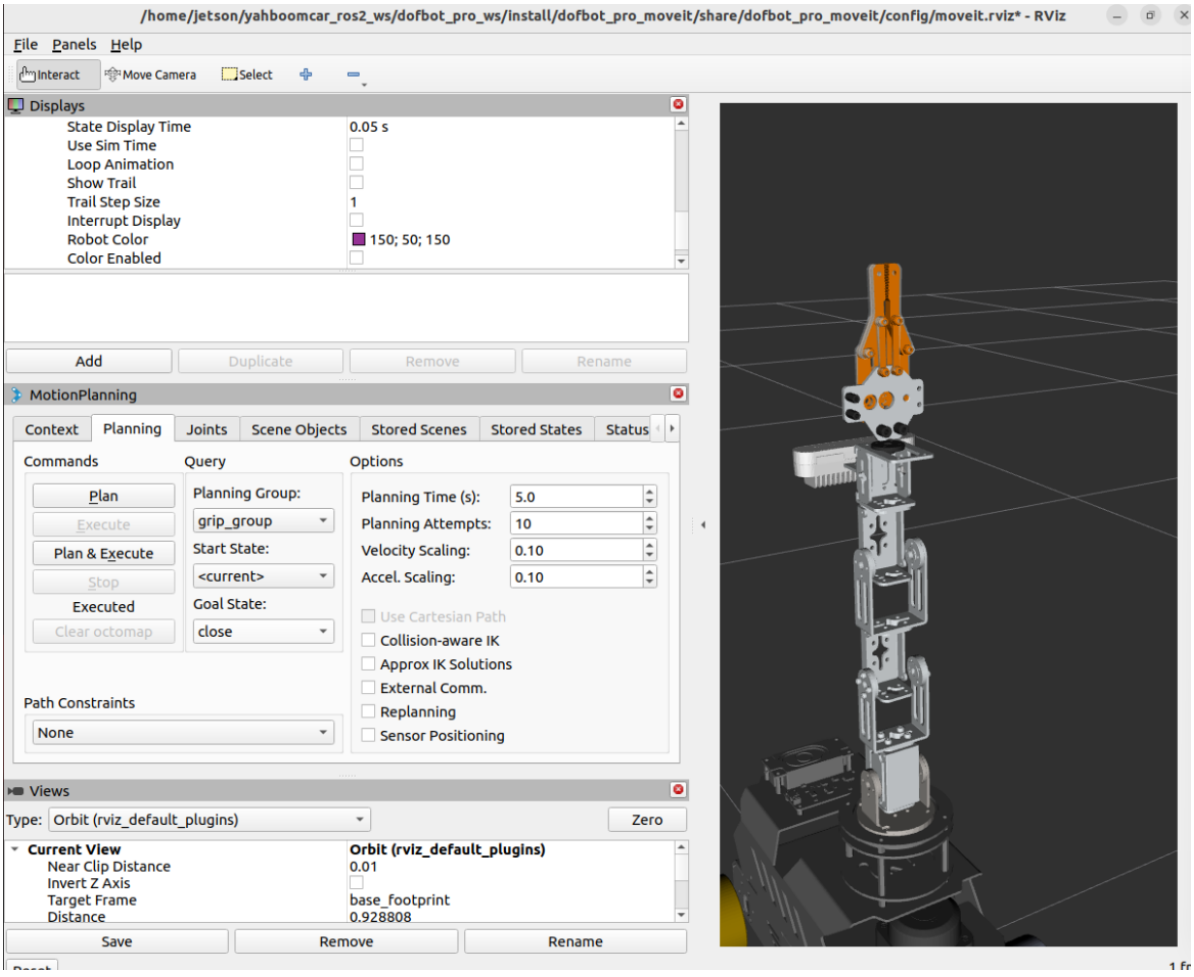
After the motion planning is complete, as shown in the figure below:



If the planning and execution are successful, it means the arm_group is configured correctly. Next, test grip_group. Configure it as shown below: set 【Planning Group】 to grip_group, 【Start State】 to , and 【Goal State】 to :



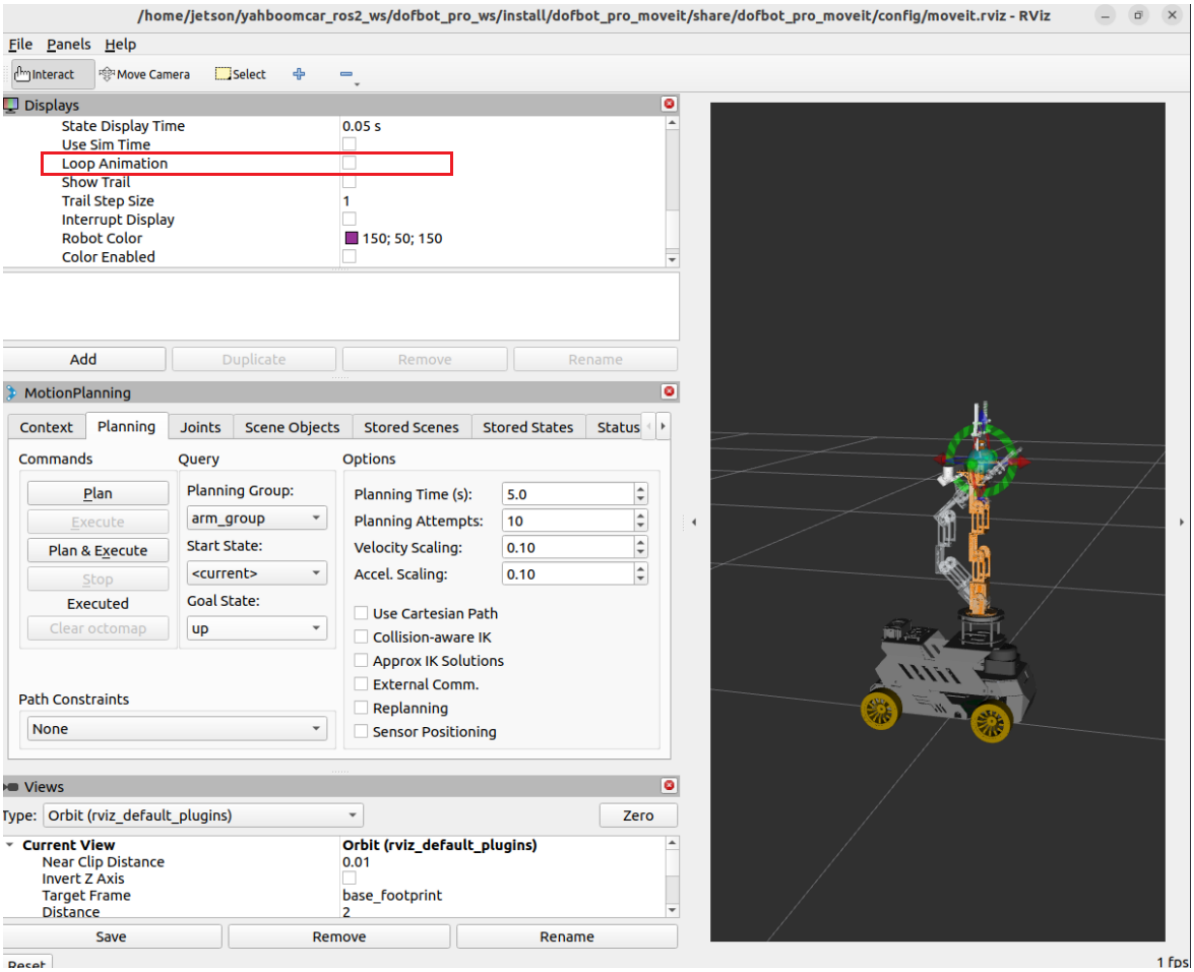
Click 【Plan & Execute】 on the left to let the gripper plan and move to the close state, as shown in the figure below:



If the gripper successfully closes, it means the planning and execution were successful.

4. Configuring RViz

When running MoveIt2, the robot arm may repeatedly execute the planning operation. This is because **loop animation** is enabled. We just need to disable this option. In 【Display】, find 【MotionPlanning】 -> 【Planned Path】 -> 【Loop Animation】, and click the checkmark here to cancel loop animation.



When running MoveIt2, the orange part represents the goal pose. If we do not want to display the goal pose, we can find 【MotionPlanning】 -> 【Query Goal State】 and click the checkmark here to cancel displaying the goal pose.

After modifying the rviz settings here, you need to **press ctrl and s to save**.